SYSEN 5151 Foundations of Systems Engineering

EcoCommute (Team 6)

Architecture Definition: Form (Physical Architecture)

First-Level Form Architecture (Chapter 8) and Second-Level Form Architecture and Component Specification (Chapter 9)

Report section combining the Chapter 8 and Chapter 9 assignments

Innoslate model: EcoCommute 11.3 (11.0 plus the physical architecture of this section) | October 2026

Team Contribution

The table lists who contributed which part of this assignment. Roles come from the team charter in the repository README.

Team member	Role	Contribution to Chapters 8 and 9
Luyu Chen	Project Manager and Integration Lead	Integrated the report and the Innoslate model (EcoCommute 11.3); Section 3 requirements traceability and spider diagram; consistency check of every figure, table and identifier across the report.
Mingjie Fang	Quality, Risk and Documentation Lead	Architecture evaluation against the functional architecture (Section 2.3) and the assessment-criteria checks (Appendix A); report editing.
Karen Li	Product and Stakeholder Lead	Choice of the decomposed function and its complexity analysis (Section 2.1.1.3); non-functional considerations (Section 2.2.1.3); alternative physical architectures and their evaluation (Section 2.4).
Yinping Yin	Sustainability Data and Analysis Lead	System design output requirements and the Reference Data Subsystem component specification, including MOPs, verification criteria and the apportionment budgets (Section 3.2).
Yiyang Zou	Technical Architecture Lead	First-level and second-level IDEF0 with mechanisms, asset hierarchy and asset diagrams, N2 and I2 diagrams, sequence diagrams (Sections 2.1 and 2.2).

Table of Contents

Team Contribution 1
1 Introduction 3
1.1 Purpose of Design Definition 3
1.2 Description of Design Definition 3
2 Architecture Definition 4
2.1 Functional Architecture 4
2.1.1 Functional Architecture Model 4
2.1.1.1 Function Elements 4
2.1.1.2 ICOM Summary for the First-Level and Second-Level Models 5
2.1.1.3 Choice of the Function to Decompose 5
2.1.1.4 Decomposition of F.2.2 into Second-Level Functions 6
2.1.2 Functional Interface 7

2.1.2.1 N2 Diagram 7
2.1.2.2 Input/Output Elements 8
2.1.3 Sequence of Operations 9
2.1.3.1 Scenario Overview 9
2.1.3.2 Detailed Sequence of Interactions 9
2.1.3.3 Consistency with IDEF0 and the N2 Diagram 11
2.2 Physical Architecture 12
2.2.1 Physical Architecture Model 12
2.2.1.1 Physical Elements 12
2.2.1.2 Asset Hierarchy Diagrams 17
2.2.1.3 Non-Functional Considerations 20
2.2.1.4 Insights on the Mechanisms 21
2.2.2 Physical Interface 21
2.2.2.1 Asset Diagrams and I-Square Diagram 21
2.3 Architecture Evaluation 26
2.3.1 Analysis against Functional Architecture 26
2.4 Alternative Physical Architectures 27
2.4.1 Evaluation of Physical Alternatives 27
3 System Requirements Management 28
3.1 Requirements Traceability 28
3.2 System Design Output Requirements 30
3.2.1 Methodology for Deriving System Design Output Requirements 30
3.2.2 Detailed System Design Output Requirements 31
3.2.2.1 Subsystem Design Output Requirements 31
3.2.2.2 Component Specification for the Reference Data Subsystem 31
3.2.2.3 System-Wide Requirements Flow-Down 33
3.2.3 Summary Table of System Design Output Requirements 36
4 Summary 38
Appendix A. Model Assessment Check 38
Appendix B. Model Baseline and Figure Sources 39

1 Introduction

1.1 Purpose of Design Definition

Chapters 6 and 7 gave EcoCommute a function architecture that was traced against the operational concept. Design Definition asks the next question: what are the parts that perform these functions, how do they connect, and what must each part do and meet? Chapter 8 allocates each first-level function to a first-level physical asset (subsystem), draws the asset diagrams that show the interfaces between assets, and completes the system-level requirements. Chapter 9 repeats the step one level lower for the most complex first-level function: it decomposes the function, allocates the new functions to second-level assets (modules), writes a component specification, and flows the system-wide requirements down to the component.

The result is a form (physical) architecture in which every function has exactly one performer, every input/output crosses a named interface, and every requirement can be given to a named asset. This is what a design team or supplier needs before building, and what the verification plan needs before testing.

1.2 Description of Design Definition

The system of interest is C.0 EcoCommute, a responsive web application that estimates commute CO2e and suggests feasible greener options; its external assets are X.1 Commuter, X.2 Pilot Administrator, X.3 Open-Meteo, X.4 Emission Factor Publisher and X.5 Hosting Provider; models 10.9 and 11.0 added X.6 Route Service and X.7 Transit Schedule Publisher for the trip planner. The work starts from Innoslate model 11.0 (the function architecture of Chapters 6 and 7, unchanged by 11.0: Metafunction, eight first-level functions F.2.0 to F.2.7 and 22 input/outputs; plus the use cases UC.3 and UC.4) and produces model 11.3. The export is stored with the report (docs/model/EcoCommute_11.3_C8-C9_physical_architecture.xml); Appendix B gives the model baseline and the source of every figure.

Section 2 presents the architecture: the functional architecture it rests on (2.1), the physical architecture with mechanisms, asset hierarchy and interfaces (2.2), its evaluation against the functional architecture (2.3) and the physical alternatives that were considered (2.4). Section 3 presents requirements management: traceability (3.1) and the system design output requirements, including the component specification and the flow-down of system-wide requirements (3.2). Table 1 lists the assignment items and where each is covered.

Table 1. Assignment items and where they are covered in this report.

Item	Activity	Source	Section
First-level physical architecture	Add mechanisms to the first-level IDEF0; create asset hierarchy diagrams	Innoslate model 11.3	2.2.1
Asset diagrams	Generate the context and first-level asset diagrams with transfers relationships	Innoslate model 11.3	2.2.2
System-level specification	Update system requirements; add system-wide requirements; trace them to C.0	Innoslate model 11.3	2.2.1.3, 3.1
Second-level functional architecture	Decompose F.2.2 Manage Reference Data into F.2.2.1 to F.2.2.4; allocate input/outputs	Innoslate model 11.3	2.1.1, 2.1.2
Second-level physical architecture	Add mechanisms C.0.2.1 to C.0.2.4; create asset hierarchy diagrams	Innoslate model 11.3	2.2.1
Component specification	Input, output, interface and functional requirements for C.0.2	Innoslate model 11.3	3.2.2
System-wide flow-down	Equivalence, apportionment or synthesis, with rationale	Innoslate model 11.3	3.2.2
Physical interface	Create I2 diagrams	Innoslate model 11.3	2.2.2.1
Team contribution	State which student did which part	Team	Front matter

Assumptions and conventions.

- Function names are verb phrases; asset and input/output names are noun phrases. Numbering: F.2.k for first-level functions, C.0.k for the subsystem that performs F.2.k, F.2.2.m for second-level functions and C.0.2.m for the module that performs F.2.2.m. The numbers mirror the hierarchy, so a function and its mechanism share their last digits. Input/outputs IO.1 to IO.22 are those of model 10.8; IO.23 and IO.24 are new.
- The one-to-one rule of the lab applies at every level: each function is performed by exactly one asset and each asset performs exactly one function. Where several hardware or software items serve one function, they are kept inside one aggregate subsystem and split at the next level.
- The four modules of the Chapter 2 walking skeleton (C.1 Web Front End, C.2 Application Service, C.3 Data Adapter, C.4 Explanation Model) are a code view. They cross function boundaries, so they are not the first-level form architecture. They are kept in the model, detached from C.0, and allocated to the new subsystems in Section 2.2.1.1 and evaluated as Alternative B in Section 2.4.
- Budgets and dates in the flow-down (cost shares, response-time shares, availability shares, the integration date) are educated guesses, as the lab allows. Each is stated with its rationale and a recommendation for detailed analysis.
- The first-level N2 diagram and the action diagram come from the Chapter 6 and 7 report (model 10.8) and are reused unchanged. Every diagram of the new architecture is an Innoslate screenshot of the model, except the two sequence diagrams, which Innoslate cannot draw with internal interactions, and any figure marked in Appendix B as drawn from the model data.

2 Architecture Definition

2.1 Functional Architecture

The form architecture is allocated from the functional architecture, so this section recalls the first level from Chapter 7 and adds the second level that Chapter 9 requires.

2.1.1 Functional Architecture Model

2.1.1.1 Function Elements

The model has functions on three levels: the Metafunction F.0 with its six top-level functions F.1 to F.6; the eight first-level functions F.2.0 to F.2.7 that decompose F.2; and the four second-level functions F.2.2.1 to F.2.2.4 that decompose F.2.2. Table 2 lists them with the asset that performs each. All functions are leaf nodes except F.0, F.2 and F.2.2.

Table 2. Function elements and the asset that performs each (model 11.3).

ID	Function	Performed by (mechanism)	Note
F.0	Metafunction	Universe	Holds the top-level functions of the system of interest and of every external asset.
F.1	Use EcoCommute Decision Support	X.1 Commuter	Top level.
F.2	Provide Commute Impact Decision Support	C.0 EcoCommute	Decomposed into F.2.0 to F.2.7.
F.3	Administer Pilot Program	X.2 Pilot Administrator	Top level.
F.4	Provide Weather Forecasts	X.3 Open-Meteo	Top level.
F.5	Publish Emission Factors	X.4 Emission Factor Publisher	Top level.
F.6	Provide Hosting Service	X.5 Hosting Provider	Top level.
F.2.0	Operate Service	C.0.0 Operations Subsystem	Leaf function.
F.2.1	Collect Commute Entries	C.0.1 Entry Subsystem	Leaf function.
F.2.2	Manage Reference Data	C.0.2 Reference Data Subsystem	Decomposed into F.2.2.1 to F.2.2.4.
F.2.3	Estimate Commute Emissions	C.0.3 Estimation Subsystem	Leaf function.

ID	Function	Performed by (mechanism)	Note
F.2.4	Manage Commute Records	C.0.4 Records Subsystem	Leaf function.
F.2.5	Evaluate Feasible Alternatives	C.0.5 Recommendation Subsystem	Leaf function.
F.2.6	Present Commute Results	C.0.6 Presentation Subsystem	Leaf function.
F.2.7	Summarize Pilot Results	C.0.7 Reporting Subsystem	Leaf function.
F.2.2.1	Load Emission Factors	C.0.2.1 Factor Import Module	Leaf function.
F.2.2.2	Select Emission Factor	C.0.2.2 Factor Lookup Module	Leaf function.
F.2.2.3	Request Weather Forecast	C.0.2.3 Weather Client Module	Leaf function.
F.2.2.4	Resolve Trip Forecast	C.0.2.4 Forecast Resolver Module	Leaf function.

2.1.1.2 ICOM Summary for the First-Level and Second-Level Models

Table 3 summarises the inputs, controls, outputs and mechanism (ICOM) of each first-level function, taken from the first-level IDEF0 of model 10.8 and completed with the mechanism that this chapter allocates. Table 4 does the same for the second-level functions. All 11 input/outputs of the Metafunction appear at the first level, and all six first-level flows of F.2.2 appear at the second level, so no input, control or output arrow is tunneled or left unconnected.

Table 3. ICOM of the first-level functions (input/output numbers; names in Table 7).

Function	Inputs	Controls	Outputs	Mechanism
F.2.0 Operate Service	IO.11	IO.9	IO.10, IO.21, IO.22	C.0.0 Operations Subsystem
F.2.1 Collect Commute Entries	IO.1	IO.2	IO.3, IO.12, IO.13	C.0.1 Entry Subsystem
F.2.2 Manage Reference Data	IO.5, IO.12	IO.6	IO.4, IO.14, IO.15	C.0.2 Reference Data Subsystem
F.2.3 Estimate Commute Emissions	IO.12	IO.14	IO.16, IO.17	C.0.3 Estimation Subsystem
F.2.4 Manage Commute Records	IO.13, IO.16	IO.2, IO.12, IO.21	IO.18, IO.19	C.0.4 Records Subsystem
F.2.5 Evaluate Feasible Alternatives	IO.12, IO.15	IO.2	IO.20	C.0.5 Recommendation Subsystem
F.2.6 Present Commute Results	IO.16, IO.18, IO.20	IO.17	IO.3	C.0.6 Presentation Subsystem
F.2.7 Summarize Pilot Results	IO.7, IO.19, IO.22	IO.9	IO.8	C.0.7 Reporting Subsystem

Table 4. ICOM of the second-level functions of F.2.2. IO.23 and IO.24 are new internal input/outputs.

Function	Inputs	Controls	Outputs	Mechanism
F.2.2.1 Load Emission Factors	none	IO.6	IO.23	C.0.2.1 Factor Import Module
F.2.2.2 Select Emission Factor	IO.12	IO.23	IO.14	C.0.2.2 Factor Lookup Module
F.2.2.3 Request Weather Forecast	IO.12	none	IO.4, IO.24	C.0.2.3 Weather Client Module
F.2.2.4 Resolve Trip Forecast	IO.5	IO.24	IO.15	C.0.2.4 Forecast Resolver Module

F.2.2.1 has a control and no input because the reviewed factors govern what the store may hold and nothing is transformed; F.2.2.3 has an input and no control because its behaviour is fixed by interface requirement 3.2.3, a constraint rather than a flow. Both are valid IDEF0 and each function still has an input or a control, an output and a mechanism.

2.1.1.3 Choice of the Function to Decompose

The lab asks the team to decompose its most complex first-level function. Table 5 scores each first-level function by the number of distinct input/outputs it touches, the number of first-level functions and external assets it exchanges information with, and the number of its interactions in the baseline sequence of UC.1 (14 interactions). The score is a simple sum so that every term can be checked in the model.

Table 5. Complexity of the first-level functions.

Function	Input/outputs	Internal partners	External assets	UC.1 interactions	Score
F.2.0 Operate Service	5	2	2	0	9
F.2.1 Collect Commute Entries	5	4	1	4	14
F.2.2 Manage Reference Data	6	3	2	6	**17**
F.2.3 Estimate Commute Emissions	4	4	0	4	12
F.2.4 Manage Commute Records	7	5	1	3	16
F.2.5 Evaluate Feasible Alternatives	4	3	1	3	11
F.2.6 Present Commute Results	5	3	1	5	14
F.2.7 Summarize Pilot Results	5	2	1	0	8

F.2.2 Manage Reference Data scores highest (17), narrowly ahead of F.2.4 Manage Commute Records (16). It is the only first-level function that exchanges information with two external assets (Open-Meteo and the Emission Factor Publisher), it takes part in six of the 14 interactions of the baseline scenario, and it feeds three other functions. F.2.4 has more input/outputs, but several of them are controls from the operational and retirement behaviour outside UC.1, so it exercises the baseline scenario less. F.2.2 also gives the component specification external interface requirements for two external systems, which is what the Chapter 9 criteria test.

2.1.1.4 Decomposition of F.2.2 into Second-Level Functions

F.2.2 holds reviewed emission factors, selects the applicable factor and obtains the forecast for the trip. It splits into four functions along two independent threads: the factor thread (load, then select) and the forecast thread (request, then resolve). The capabilities are:

- **F.2.2.1 Load Emission Factors.** Reads a published factor file or source document, rejects sets that lack source, units, geography, methodology or version, and stores the accepted set under a new version.
- **F.2.2.2 Select Emission Factor.** Chooses the applicable factor for the trip mode and geography from the versioned set and returns it with its provenance.
- **F.2.2.3 Request Weather Forecast.** Builds the HTTPS request from the coarse area and trip time only, sends it to Open-Meteo with a timeout.
- **F.2.2.4 Resolve Trip Forecast.** Picks the forecast hour that matches the trip, keeps the forecast timestamp and flags a missing or late response.

The six input/outputs carried down from F.2.2 are allocated as follows: IO.6 (control) to F.2.2.1; IO.12 (input) to F.2.2.2 and F.2.2.3; IO.5 (input) to F.2.2.4; IO.14 from F.2.2.2, IO.4 from F.2.2.3 and IO.15 from F.2.2.4. Two internal flows are needed to complete the baseline use case: IO.23 Versioned Factor Set from F.2.2.1 to F.2.2.2 (a control that tells the selection which factors exist), and IO.24 Forecast Query Context from F.2.2.3 to F.2.2.4 (a control that tells the resolver which place and hour to pick out of the response). The two threads do not exchange information with each other, which is why the decomposition is cohesive. Table 6 records the third-level decomposition planned for the final assignment, so that the most complex second-level function can be broken down once more.

Table 6. Planned third-level decomposition (not modelled in this section).

Second-level function	Planned third-level functions
F.2.2.1 Load Emission Factors	F.2.2.1.1 Read Factor Source, F.2.2.1.2 Validate Factor Attributes, F.2.2.1.3 Version Factor Set
F.2.2.3 Request Weather Forecast	F.2.2.3.1 Build Weather Request, F.2.2.3.2 Send Weather Request

2.1.2 Functional Interface

2.1.2.1 N2 Diagram

Figure 1 is the N2 diagram of the first-level functions from model 10.8. The functions sit on the diagonal; a cell above or to the right of the diagonal holds the input/output that the function in its row sends to the function in its column; the top row holds what enters from outside and the right-hand column what leaves. Figure 2 is the N2 diagram of the four second-level functions: only two internal cells are filled (IO.23 and IO.24), which shows that the two threads are decoupled, and the top row and right-hand column hold the six boundary flows.

IO.9 Pilot Program Rules IO.11 Hosting Service Status	IO.1 Commuter Entries and IO.2 Commuter Consent and	IO.5 Weather Forecast Data IO.6 Reviewed Emission		IO.2 Commuter Consent and	IO.2 Commuter Consent and		IO.7 Aggregate Summary IO.9 Pilot Program Rules	
F.2.0 Operate Service				IO.21 Retention Directive			IO.22 Service Status Record	IO.10 Release Configuration
	F.2.1 Collect Commute Entries	IO.12 Validated Commute	IO.12 Validated Commute	IO.12 Validated Commute IO.13 Record Deletion	IO.12 Validated Commute			IO.3 Commute Result and
		F.2.2 Manage Reference Data	IO.14 Emission Factor		IO.15 Trip Weather Forecast			IO.4 Weather Request
			F.2.3 Estimate Commute Emissions	IO.16 Trip Emission Estimate		IO.16 Trip Emission Estimate IO.17 Estimation Method		
				F.2.4 Manage Commute Records		IO.18 Commute Trend	IO.19 Trip Record Extract	
					F.2.5 Evaluate Feasible Alternatives	IO.20 Alternative		
						F.2.6 Present Commute Results		IO.3 Commute Result and
							F.2.7 Summarize Pilot Results	IO.8 Approved Aggregate

Figure 1. N2 diagram of the first-level functions from Innoslate (model 10.8).

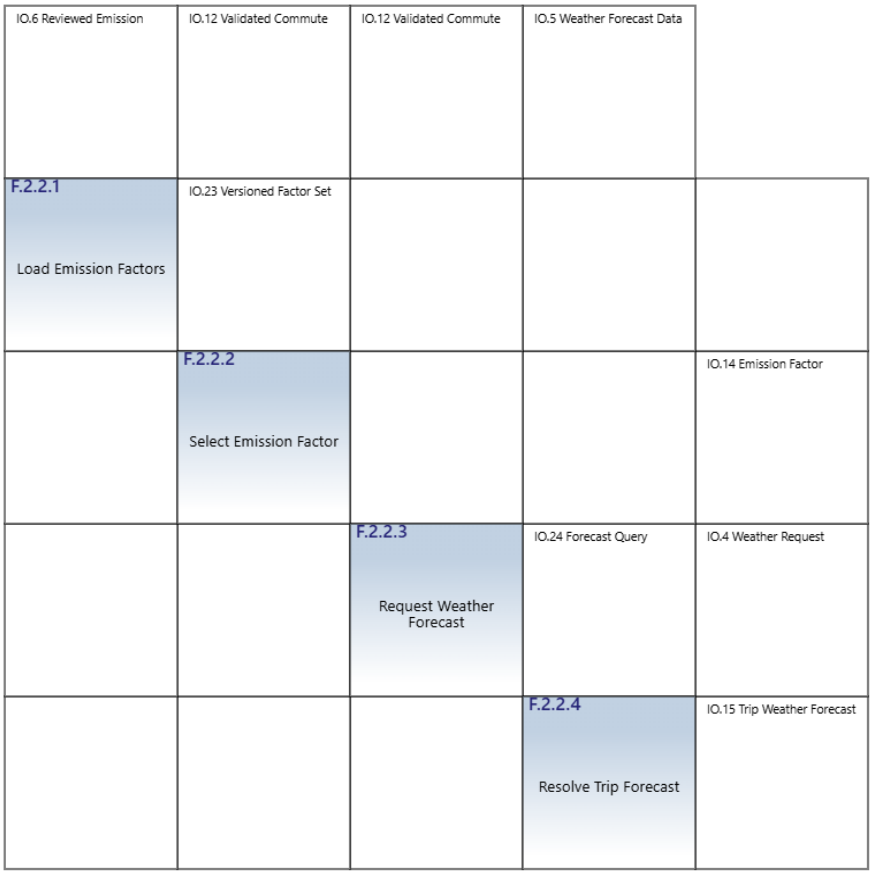


Figure 2. N2 diagram of the second-level functions of F.2.2 (model 11.3).

2.1.2.2 Input/Output Elements

Table 7. Input/outputs of the model. IO.1 to IO.11 are carried down from the Metafunction, IO.12 to IO.22 are first-level internal, IO.23 and IO.24 are second-level internal.

ID	Name	Level introduced
IO.1	Commuter Entries and Requests	Metafunction
IO.2	Commuter Consent and Settings	Metafunction
IO.3	Commute Result and Advice	Metafunction
IO.4	Weather Request	Metafunction
IO.5	Weather Forecast Data	Metafunction
IO.6	Reviewed Emission Factors	Metafunction
IO.7	Aggregate Summary Request	Metafunction
IO.8	Approved Aggregate Summary	Metafunction
IO.9	Pilot Program Rules	Metafunction
IO.10	Release Configuration	Metafunction
IO.11	Hosting Service Status	Metafunction
IO.12	Validated Commute Entry	first level
IO.13	Record Deletion Request	first level
IO.14	Emission Factor Record	first level
IO.15	Trip Weather Forecast	first level
IO.16	Trip Emission Estimate	first level
IO.17	Estimation Method Disclosure	first level
IO.18	Commute Trend Summary	first level
IO.19	Trip Record Extract	first level
IO.20	Alternative Recommendation	first level
IO.21	Retention Directive	first level
IO.22	Service Status Record	first level

ID	Name	Level introduced
IO.23	Versioned Factor Set	second level (new)
IO.24	Forecast Query Context	second level (new)

2.1.3 Sequence of Operations

2.1.3.1 Scenario Overview

The scenario is the baseline use case UC.1, Review Commute Impact and Feasible Alternative: a Commuter with an account and consent opens the entry form, submits a valid trip, and receives the estimated CO2e, the difference from the always-drive baseline, the method disclosure, a feasible alternative and an explanation. It assumes suitable stored emission factors and an available forecast. The action diagram of UC.1 (Figure 3) fixes the order of the six external input/outputs that the sequence must reproduce.

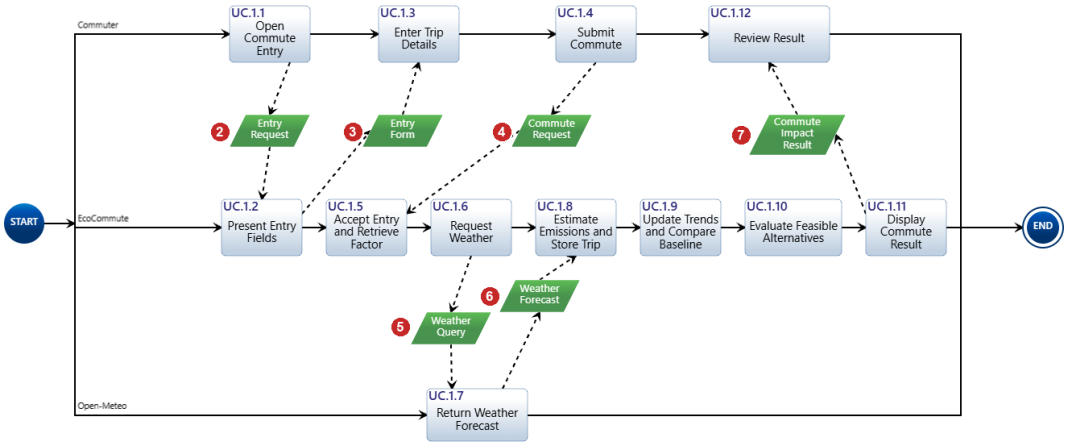


Figure 3. UC.1 action diagram from Innoslate with the external input/outputs numbered 2 to 7 in order of occurrence.

Table 8 checks that every input/output of the action diagram is accounted for by an input/output of the Metafunction IDEF0. All six are; the seventh interaction of the scenario, Reviewed Emission Factors (IO.6), is a precondition that happens before the use case starts and is therefore not on the action diagram.

Table 8. Input/outputs of the UC.1 action diagram and the Metafunction input/output that accounts for each.

Order	Action diagram input/output	Metafunction input/output and path	Result
2	Entry Request	IO.1 Commuter Entries and Requests: F.1 > F.2	Found
3	Entry Form	IO.3 Commute Result and Advice: F.2 > F.1	Found (F.2.1 added as a source of IO.3 in model 10.8)
4	Commute Request	IO.1 Commuter Entries and Requests: F.1 > F.2; IO.2 Commuter Consent and Settings: F.1 > F.2	Found
5	Weather Query	IO.4 Weather Request: F.2 > F.4	Found (same flow, different name)
6	Weather Forecast	IO.5 Weather Forecast Data: F.4 > F.2	Found (same flow, different name)
7	Commute Impact Result	IO.3 Commute Result and Advice: F.2 > F.1	Found

2.1.3.2 Detailed Sequence of Interactions

Figure 4 replays UC.1 over the first-level assets. Each interaction is numbered once and named once; red interactions cross the system boundary and blue interactions are internal. Figure 5 zooms into the Reference Data

Subsystem and replays the eight interactions of the scenario that cross or stay inside C.0.2. A flow that reaches several functions (IO.12 to four subsystems) is one interaction with several arrow heads.

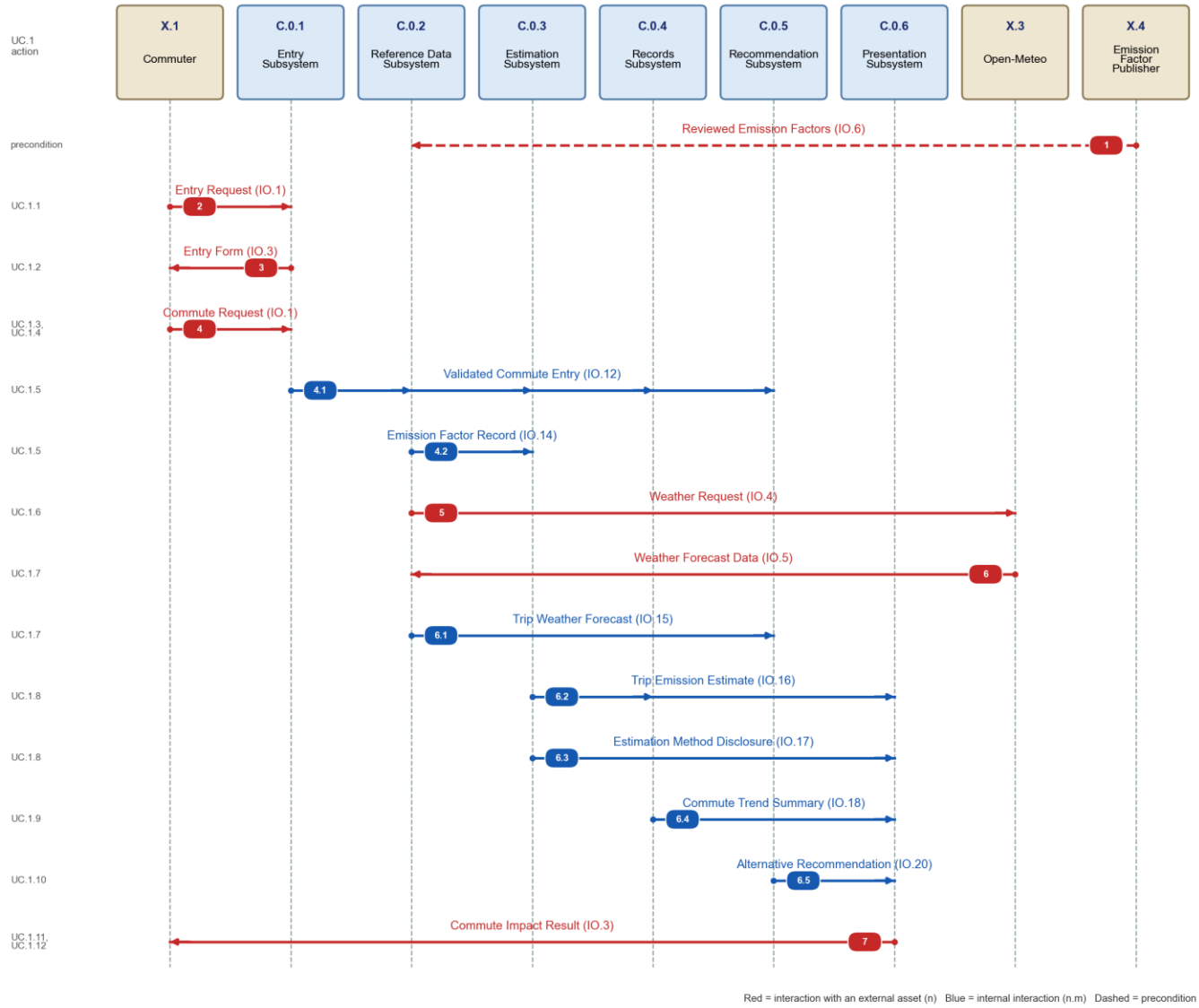


Figure 4. UC.1 sequence diagram over the first-level assets (model 11.3). Lifelines are the assets that perform the first-level functions.

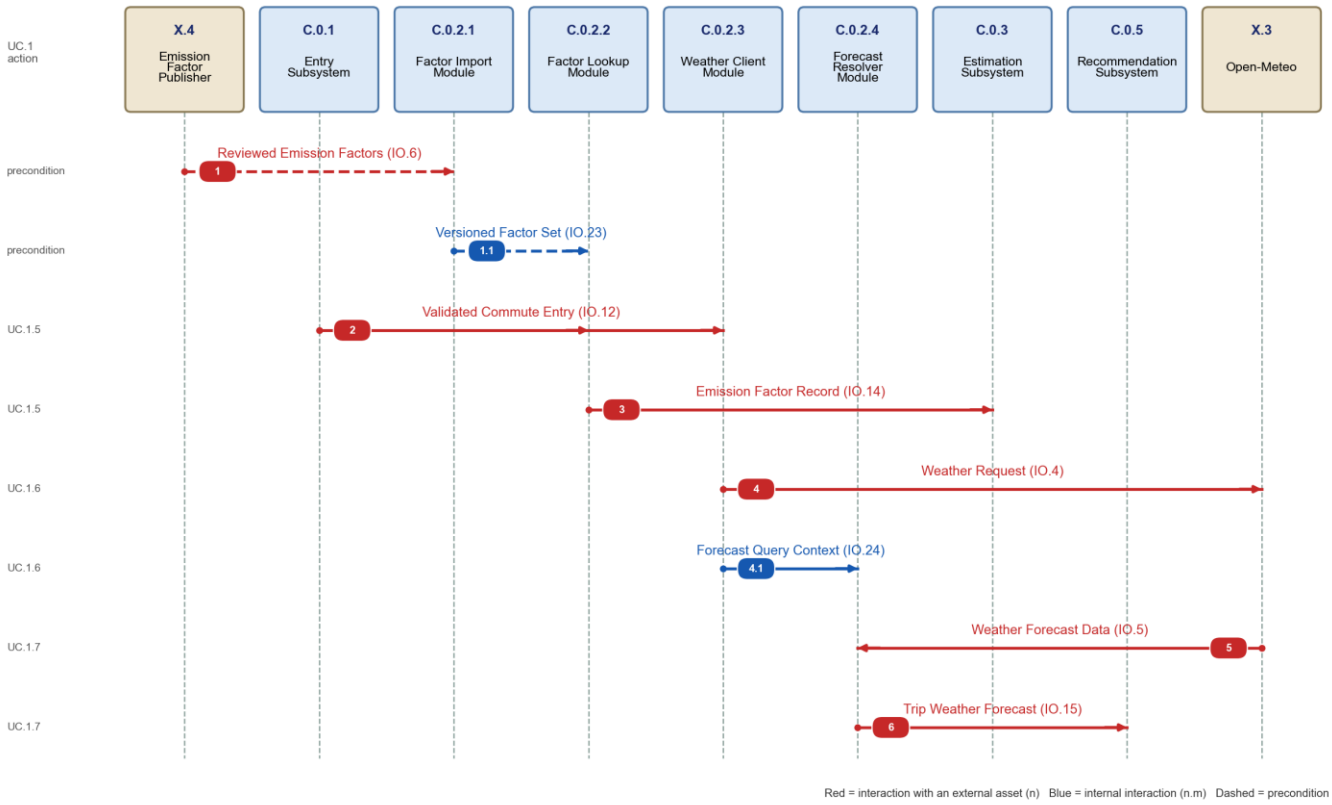


Figure 5. UC.1 sequence diagram over the second-level assets of C.0.2 (model 11.3).

2.1.3.3 Consistency with IDEF0 and the N2 Diagram

Table 9. First-level sequence steps mapped to the IDEF0 flow and to the asset that sends and receives it. Every numbered step appears once.

Step	Interaction	Sender > receivers	IO	UC.1 action
1	Reviewed Emission Factors	X.4 > C.0.2	IO.6	precondition
2	Entry Request	X.1 > C.0.1	IO.1	UC.1.1
3	Entry Form	C.0.1 > X.1	IO.3	UC.1.2
4	Commute Request	X.1 > C.0.1	IO.1	UC.1.3, UC.1.4
4.1	Validated Commute Entry	C.0.1 > C.0.2, C.0.3, C.0.4, C.0.5	IO.12	UC.1.5
4.2	Emission Factor Record	C.0.2 > C.0.3	IO.14	UC.1.5
5	Weather Request	C.0.2 > X.3	IO.4	UC.1.6
6	Weather Forecast Data	X.3 > C.0.2	IO.5	UC.1.7
6.1	Trip Weather Forecast	C.0.2 > C.0.5	IO.15	UC.1.7
6.2	Trip Emission Estimate	C.0.3 > C.0.4, C.0.6	IO.16	UC.1.8
6.3	Estimation Method Disclosure	C.0.3 > C.0.6	IO.17	UC.1.8
6.4	Commute Trend Summary	C.0.4 > C.0.6	IO.18	UC.1.9
6.5	Alternative Recommendation	C.0.5 > C.0.6	IO.20	UC.1.10
7	Commute Impact Result	C.0.6 > X.1	IO.3	UC.1.11, UC.1.12

Table 10. Second-level sequence steps for C.0.2. Step 2 reaches two modules because IO.12 is an input to F.2.2.2 and F.2.2.3.

Step	Interaction	Sender > receivers	IO	UC.1 action
1	Reviewed Emission Factors	X.4 > C.0.2.1	IO.6	precondition
1.1	Versioned Factor Set	C.0.2.1 > C.0.2.2	IO.23	precondition
2	Validated Commute Entry	C.0.1 > C.0.2.2, C.0.2.3	IO.12	UC.1.5
3	Emission Factor Record	C.0.2.2 > C.0.3	IO.14	UC.1.5
4	Weather Request	C.0.2.3 > X.3	IO.4	UC.1.6
4.1	Forecast Query Context	C.0.2.3 > C.0.2.4	IO.24	UC.1.6
5	Weather Forecast Data	X.3 > C.0.2.4	IO.5	UC.1.7
6	Trip Weather Forecast	C.0.2.4 > C.0.5	IO.15	UC.1.7

All 14 first-level interactions and all eight second-level interactions carry an input/output that exists on the corresponding IDEF0 and appears in the corresponding cell of the N2 diagram, and every input/output of those IDEF0 diagrams that UC.1 uses appears in the sequence. The sequence is continuous: each interaction is triggered by an earlier one, from the precondition (factors loaded) through the Commute Impact Result (step 7) that ends the scenario. No input, control or output arrow is tunneled or left unconnected on the Metafunction, the first-level IDEF0 or the second-level IDEF0.

2.2 Physical Architecture

2.2.1 Physical Architecture Model

2.2.1.1 Physical Elements

Each first-level function receives one subsystem, named for the capability it provides rather than for the function (a noun, no conjunction). Table 11 lists the eight first-level subsystems and what each is made of; at this level a subsystem is still an aggregate of software, data, documents and people, as the lab describes. Table 12 lists the four modules of C.0.2. The hierarchy mirrors the functional hierarchy one to one: C.0 EcoCommute is decomposed by C.0.0 to C.0.7, as F.2 is decomposed by F.2.0 to F.2.7, and C.0.2 is decomposed by C.0.2.1 to C.0.2.4, as F.2.2 is decomposed by F.2.2.1 to F.2.2.4.

Table 11. First-level physical elements (subsystems).

Asset	Name	Performs	Made of
C.0.0	Operations Subsystem	F.2.0 Operate Service	Release package and deployment scripts, availability probe and alert rules, retirement runbook, pilot operations staff.
C.0.1	Entry Subsystem	F.2.1 Collect Commute Entries	Responsive web entry form, input validation rules, consent check, deletion-request handler.
C.0.2	Reference Data Subsystem	F.2.2 Manage Reference Data	Reviewed emission-factor store with version control, factor selection logic, Open-Meteo client and forecast matching.
C.0.3	Estimation Subsystem	F.2.3 Estimate Commute Emissions	Calculation engine for CO2e, distance convention, always-drive baseline rule and method disclosure text.
C.0.4	Records Subsystem	F.2.4 Manage Commute Records	Trip record database, access control, weekly/monthly trend aggregation, deletion and retention jobs.
C.0.5	Recommendation Subsystem	F.2.5 Evaluate Feasible Alternatives	Rule set on distance, forecast weather and feasibility settings; one alternative with a stated reason.
C.0.6	Presentation Subsystem	F.2.6 Present Commute Results	Result assembly, plain-language explanation model (restates calculated facts only), responsive result view.
C.0.7	Reporting Subsystem	F.2.7 Summarize Pilot Results	Aggregation, small-group suppression, authenticated dashboard and approved aggregate export.

Table 12. Second-level physical elements (modules of C.0.2).

Asset	Name	Performs	Made of
C.0.2.1	Factor Import Module	F.2.2.1 Load Emission Factors	Reads a published factor file or source document, rejects sets that lack source, units, geography, methodology or version, and stores the accepted set under a new version.
C.0.2.2	Factor Lookup Module	F.2.2.2 Select Emission Factor	Chooses the applicable factor for the trip mode and geography from the versioned set and returns it with its provenance.
C.0.2.3	Weather Client Module	F.2.2.3 Request Weather Forecast	Builds the HTTPS request from the coarse area and trip time only, sends it to Open-Meteo with a timeout.
C.0.2.4	Forecast Resolver Module	F.2.2.4 Resolve Trip Forecast	Picks the forecast hour that matches the trip, keeps the forecast timestamp and flags a missing or late response.

Figure 6 is the Metafunction IDEF0 with its six mechanisms (Commuter, EcoCommute, Pilot Administrator, Open-Meteo, Emission Factor Publisher, Hosting Provider). Figure 7 is the first-level IDEF0 with the mechanisms added: each box has one mechanism, its subsystem (Innoslate draws the mechanism arrows from the parent asset at the bottom and labels each arrow with the performing subsystem). Figure 8 is the second-level IDEF0 of F.2.2 with the four module mechanisms. Together the three diagrams give IDEF0 to three levels, each with inputs, controls, outputs and mechanisms.

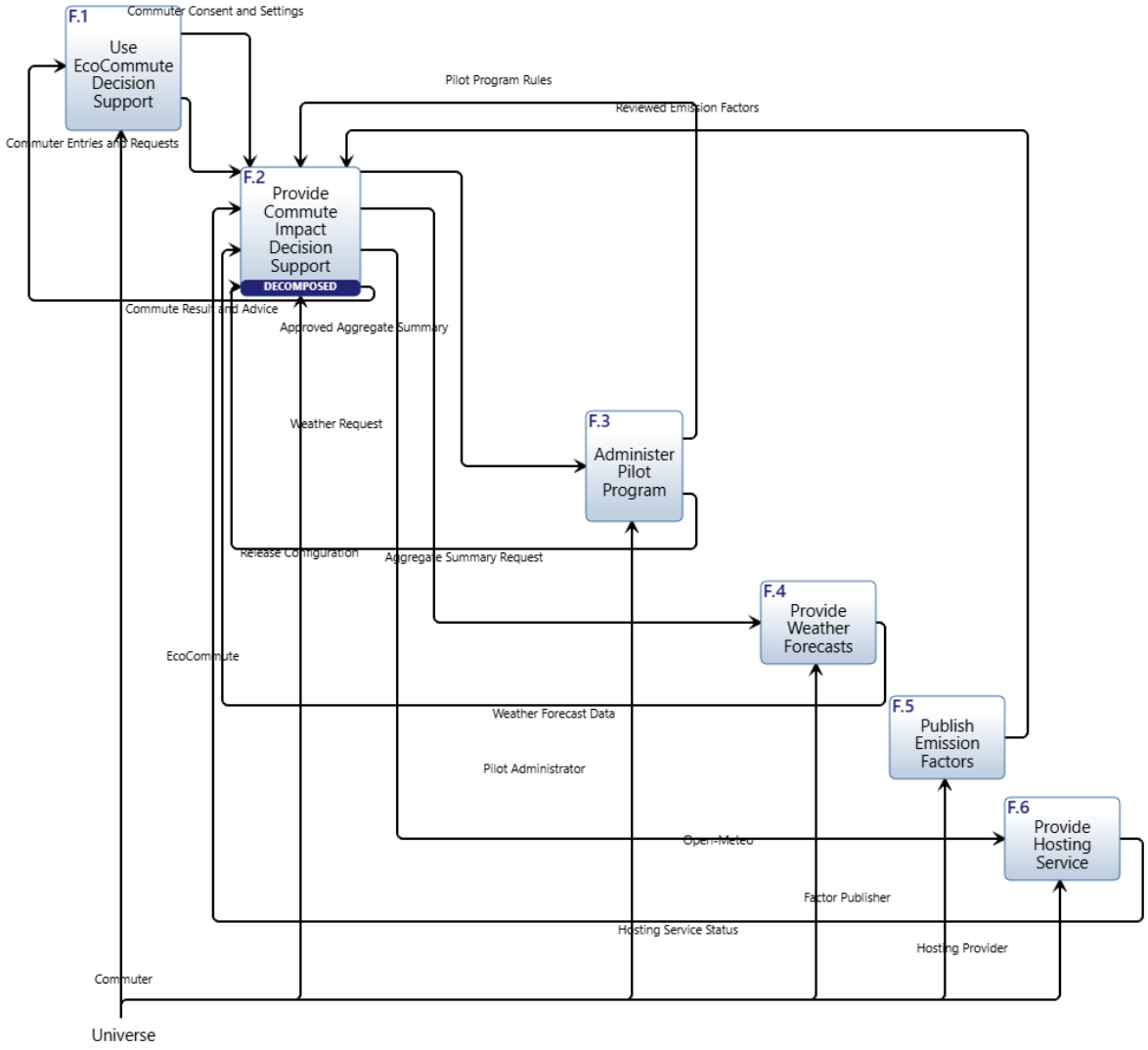


Figure 6. Metafunction IDEF0 from Innoslate: six top-level functions with their mechanisms at the bottom.

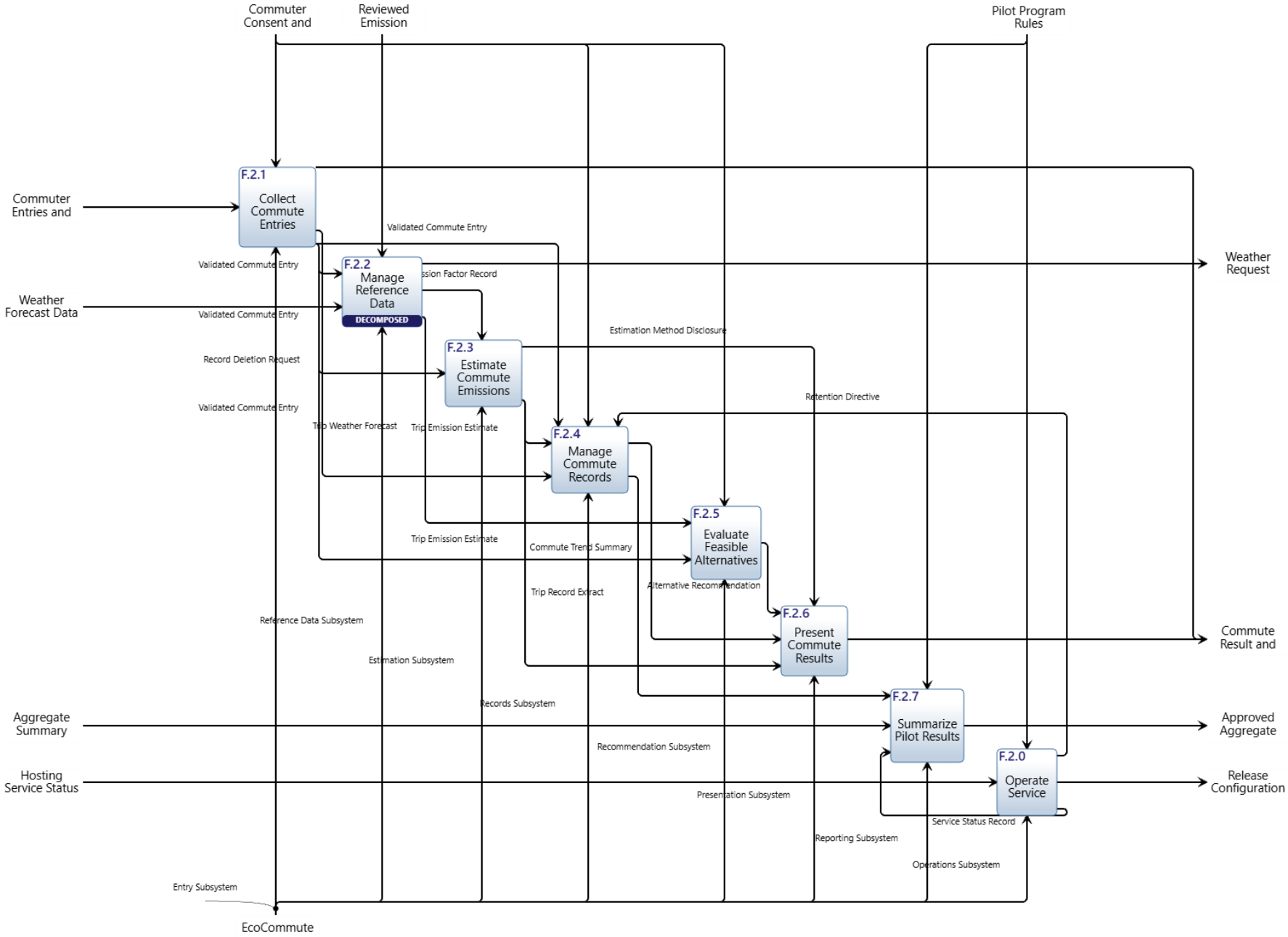


Figure 7. First-level IDEF0 of F.2 with mechanisms (model 11.3): each function is performed by one subsystem C.0.k.

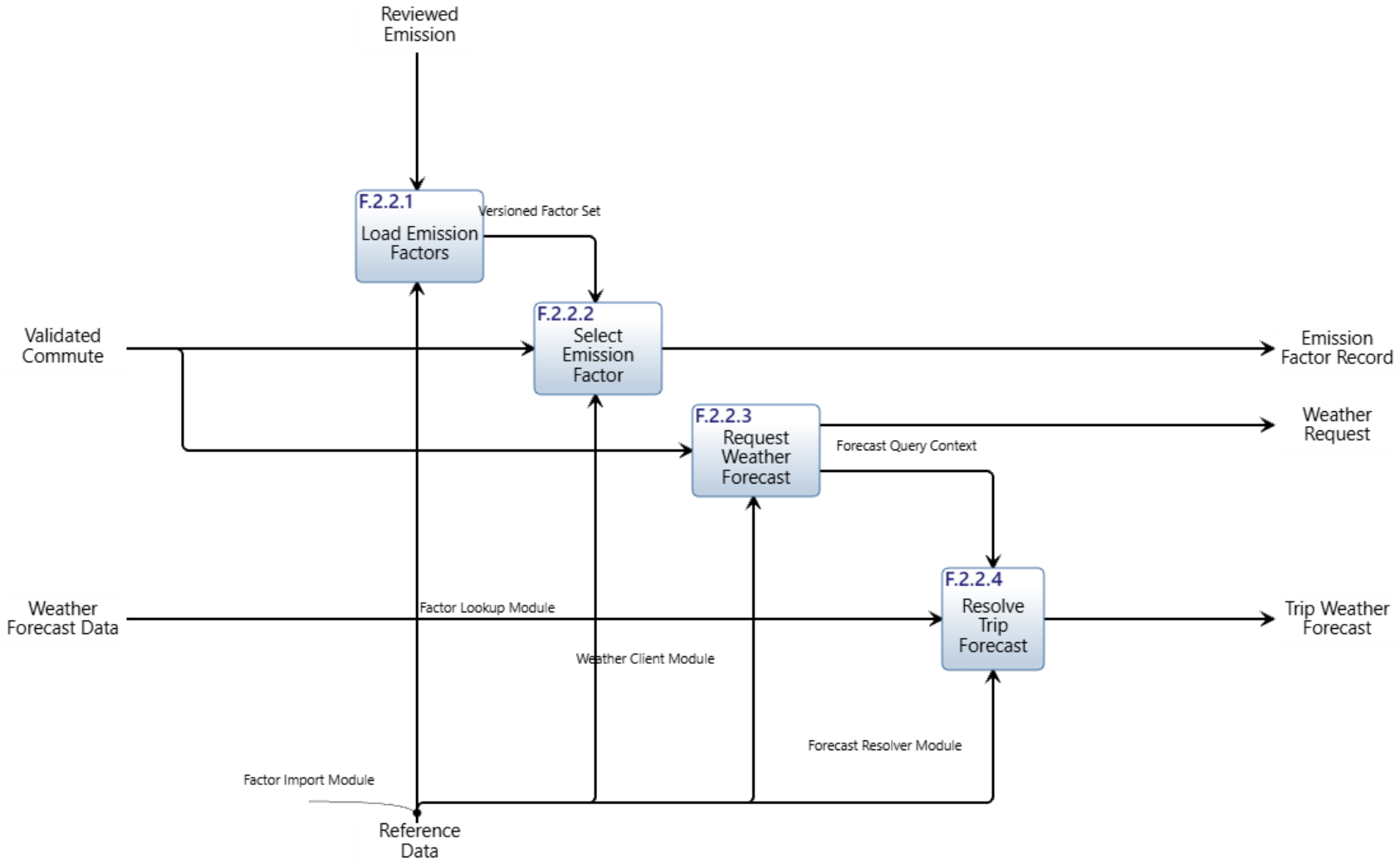


Figure 8. Second-level IDEF0 of F.2.2 with mechanisms (model 11.3): each function is performed by one module C.0.2.m.

Walking-skeleton modules. The four modules of the Chapter 2 code (C.1 to C.4) each serve more than one function, so they are not first-level assets. Their allocation to the new subsystems is below; the repository directories keep their names until the product-build increment of Chapter 8.5, when docs/deployment.md maps directories to subsystems.

Table 13. Allocation of the walking-skeleton code modules to the subsystems.

Module	Name	Implements
C.1	Web Front End	C.0.1 Entry (entry form), C.0.6 Presentation (result view)
C.2	Application Service	C.0.3 Estimation, C.0.5 Recommendation, C.0.4 Records (trend logic)
C.3	Data Adapter	C.0.2 Reference Data (factor store, weather access), C.0.4 Records (trip store)
C.4	Explanation Model	C.0.6 Presentation (explanation step)

Table 14 records the technology each subsystem is built with and the existing element it reuses, taken from the repository and its architecture decision records (ADRs). It shows that seven of the eight subsystems already have an implementation; the Reporting Subsystem does not, because the pilot summary (UC.2) lies outside the implemented use cases.

Table 14. Technologies and existing elements of the first-level subsystems.

Subsystem	Technology and existing element	Source
C.0.0 Operations	Python 3.12 application served with Waitress; invite and account management command (src.manage); hosting by X.5	ADR 0001, 0009
C.0.1 Entry	Flask web entry with validation, sign-up and log-in (C.1 web_app, auth)	ADR 0002, 0009
C.0.2 Reference Data	Factor store (placeholder factors until reviewed ones are loaded); route adapter to OpenRouteService with labelled fallback; place and route cache in SQLite; TCAT timetable index	ADR 0003, 0007, 0009
C.0.3 Estimation	CO2e calculation and mode comparison, with carpool share (C.2)	ADR 0003, 0008
C.0.4 Records	SQLite database app.sqlite for users, trips and preferences; salted scrypt hashes; badge rules (C.2 achievements)	ADR 0005, 0009
C.0.5 Recommendation	Rule-based recommendation (recommend_mode) and bus planner on the TCAT timetable (C.2)	ADR 0004, 0006, 0007
C.0.6 Presentation	Flask result pages; explanation step (C.4), currently a template that restates calculated facts	ADR 0002
C.0.7 Reporting	Not implemented yet; the campus tree total is computed in C.2	ADR 0005

2.2.1.2 Asset Hierarchy Diagrams

Figure 9 is the whole asset hierarchy to three levels below the Universe: C.0 and the seven external assets (level 1; X.6 and X.7 were added in model 11.0), the eight subsystems (level 2) and the four modules of C.0.2 (level 3). Because the full tree is too wide to read, it is partitioned: Figure 10 shows the context (Universe, C.0 and the external assets), Figure 11 shows C.0 with its eight subsystems and the function each performs, and Figure 12 shows C.0.2 with its four modules and the function each performs. Every asset has a number and a noun-oriented name, and every function-asset pair shares its last digits.

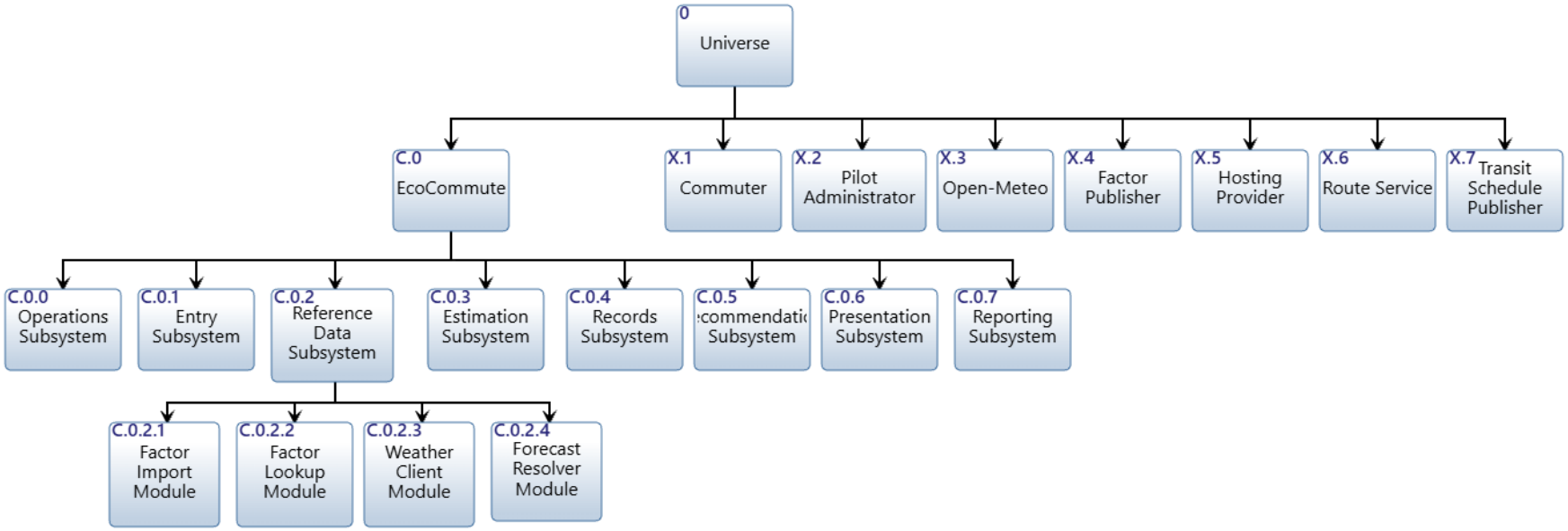


Figure 9. Overall asset hierarchy (model 11.3), three levels below the Universe.

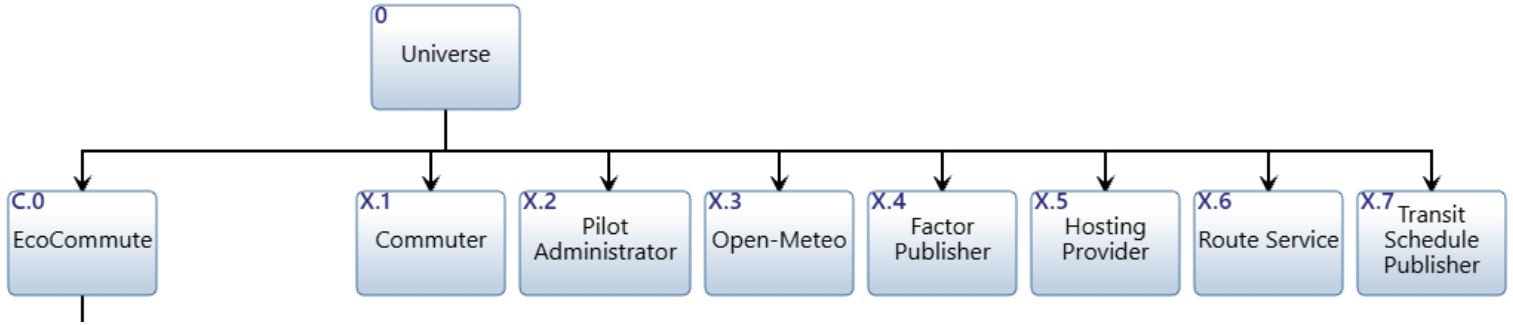


Figure 10. Hierarchy partition 0: the Universe with C.0 and the external assets X.1 to X.7.

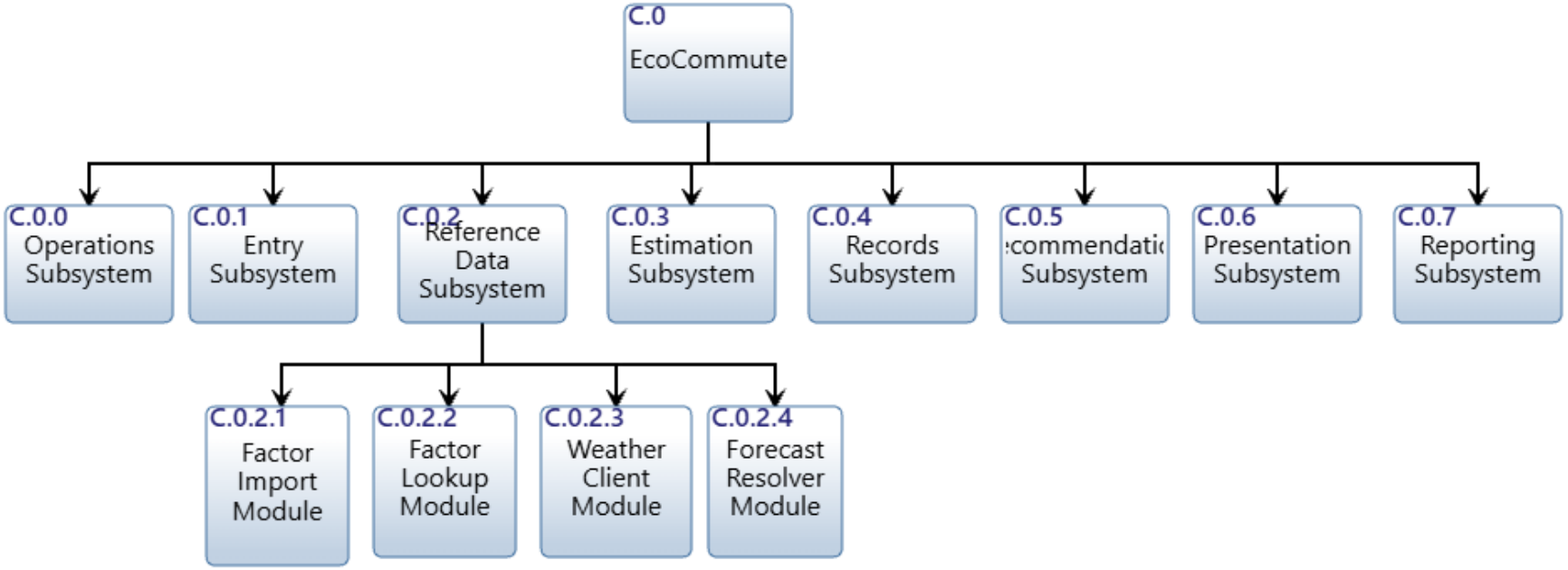


Figure 11. Hierarchy partition 1: C.0 EcoCommute decomposed into its eight first-level subsystems.

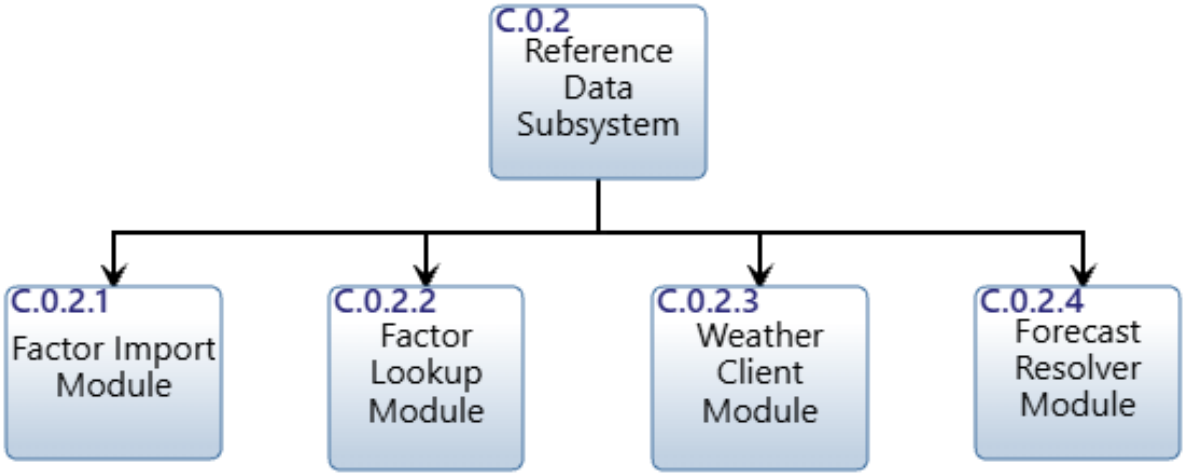


Figure 12. Hierarchy partition 2: C.0.2 Reference Data Subsystem decomposed into its four modules.

2.2.1.3 Non-Functional Considerations

System-wide (non-functional) requirements cannot be allocated to one asset, so they are satisfied by the system asset C.0 and contributed to by the subsystems. In model 11.3 each of the 17 leaf requirements of 3.4 System-wide Requirements (11 from model 10.8, six added by models 10.9 and 11.0) is traced to C.0 with a satisfied-by relationship (Chapter 8 criterion). Table 15 shows which subsystems contribute to each requirement: P marks the primary contributor, C a contributor. The matrix drives the selection of the component for flow-down in Section 3.2.2 and the weights in Section 2.4.

Table 15. Contribution of the first-level subsystems to the system-wide requirements (P primary, C contributes).

Requirement	C.0.0	C.0.1	C.0.2	C.0.3	C.0.4	C.0.5	C.0.6	C.0.7
3.4.1.1 cost no more than $750,000	C	C	C	C	C	C	C	C
3.4.1.2 report results before December 18, 2026	C		C		C			P
3.4.2.1 display each trip result within 3 seconds		C	P	C	C	C	P	
3.4.2.2 complete entry in 30 seconds or less		P						
3.4.3.1 availability of at least 99.5 percent	P	C	C	C	C	C	C	
3.4.3.2 operate with at least 500 enrolled accounts		C	C	C	P	C	C	C
3.4.4.1 conform to WCAG 2.2 Level AA		P					P	
3.4.5.1 restrict access to individual trip records		C			P			C
3.4.5.2 store only coarse location, no device traces		P	C		C			
3.4.5.3 suppress aggregate groups below 5								P
3.4.6.1 reproduce reference cases to the nearest gram			C	P				
3.4.5.4 send the Route Service only the entered places		C	P					
3.4.5.5 store passwords and invite codes only as hashes					P			
3.4.5.6 lock log-in after repeated wrong passwords		C			P			
3.4.5.7 keep no record of who asked in the place cache			P		C			
3.4.6.2 label route-based figures as estimates				P			P	
3.4.6.3 state that bus times come from a timetable						P	C	

Observations: availability and response time involve every subsystem on the UC.1 path, so they are budgeted (apportionment); privacy and quality requirements have one or two primary owners; accessibility concerns only

the two subsystems with a user interface; and the Reference Data Subsystem contributes to nine of the 17 requirements (cost, schedule, response time, availability, capacity, location privacy, factor resolution and, since model 11.0 allocated place suggestion to F.2.2, the Route Service data limit and the place-cache privacy rule), which supports choosing it for the flow-down.

2.2.1.4 Insights on the Mechanisms

Table 16. The mechanisms: what each is made of and how many conduits it has (internal conduits out and in, conduits to external assets).

Mechanism	Mostly	Out	In	External
C.0.0 Operations	software, scripts, people	2	0	2
C.0.1 Entry	software (user interface)	4	0	1
C.0.2 Reference Data	software and reference data	2	1	3
C.0.3 Estimation	software (calculation)	2	2	0
C.0.4 Records	software and personal data	2	3	1
C.0.5 Recommendation	software (rules)	1	2	1
C.0.6 Presentation	software (user interface, language step)	0	3	1
C.0.7 Reporting	software, data and people	0	2	2

Four observations about the mechanisms follow from the allocation. (1) **The mechanisms are cohesive.** Each subsystem is named for one capability and holds one function, so a change to a rule (recommendation), a factor (reference data) or a view (presentation) stays inside one asset. (2) **Interfaces concentrate in two assets.** C.0.1 Entry sends to four subsystems and C.0.4 Records receives from three subsystems and has one external conduit, so these two define most of the internal interface work. (3) **Three mechanisms touch the outside world.** Reference Data has three external conduits (Open-Meteo twice, the Emission Factor Publisher once), Operations two (Hosting Provider, Pilot Administrator) and Reporting two (both with the Pilot Administrator); the Commuter reaches four subsystems, which is the IO.2 fan-out discussed in Section 2.3.1. (4) **Two mechanisms are partly people and process.** Operations (release, monitoring, retirement) and Reporting (small-group suppression rules set by the Pilot Administrator) are not only software, so their requirements are verified by analysis and demonstration rather than test alone (Table 32).

2.2.2 Physical Interface

2.2.2.1 Asset Diagrams and I-Square Diagram

The asset diagrams show the interfaces (conduits) between assets and the input/outputs that each conduit transfers. Figure 13 is the context asset diagram generated from the Metafunction IDEF0: C.0 interfaces with the five external assets over the eight conduits of model 10.8, each with its transfers relationships; the three conduits that model 11.0 added for X.6 and X.7 are shown with the grey assets X.6 and X.7 and transfer use-case-level input/outputs (Route Request, Route Options, Bus Schedule) that have no top-level function yet. Figure 14 is the first-level asset diagram generated from the first-level IDEF0. It has 24 conduits: 11 between a subsystem and an external asset and 13 between subsystems. Table 18 lists each conduit, its endpoints, the medium and what it transfers. The 22 first-level input/outputs are all transferred by at least one conduit, and each conduit transfers only input/outputs that appear on the IDEF0 between the same two functions.

Table 17 lists the transfers relationship of each context conduit and compares it with the Metafunction IDEF0: all 11 input/outputs of the Metafunction are transferred, each between the two assets whose functions exchange it.

Table 17. Context conduits, the input/outputs they transfer and agreement with the Metafunction IDEF0.

Conduit	Direction	Transfers	Matches Metafunction
Trip details	X.1 > C.0	IO.1, IO.2	Yes
Result and advice	C.0 > X.1	IO.3	Yes
Weather query	C.0 > X.3	IO.4	Yes
Forecast	X.3 > C.0	IO.5	Yes
Reviewed factors	X.4 > C.0	IO.6	Yes
Summary request	X.2 > C.0	IO.7, IO.9	Yes
Approved aggregates	C.0 > X.2	IO.8	Yes
Hosting service	C.0 > X.5 and X.5 > C.0	IO.10, IO.11	Yes
Route request, Route response, Transit schedule	C.0 and X.6, X.7	Route Request, Route Options, Bus Schedule	Added by model 11.0; no top-level function

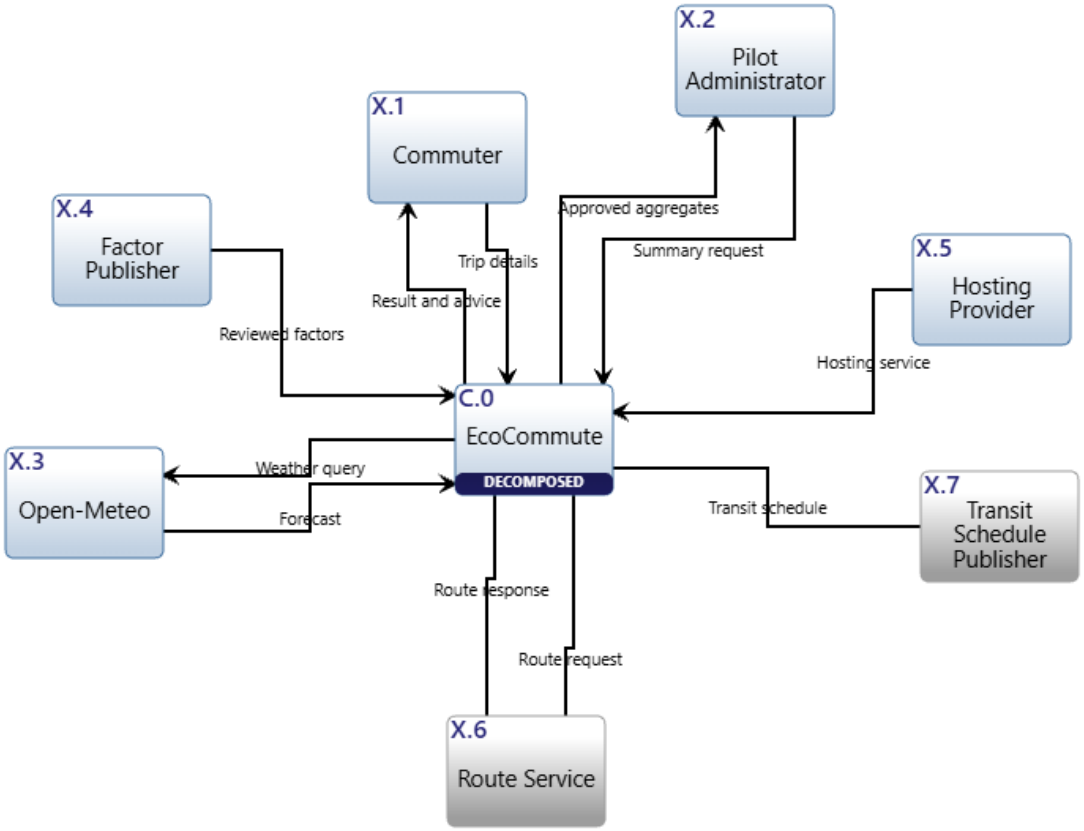


Figure 13. Context asset diagram of C.0 EcoCommute with its seven external assets (model 11.3; conduit names as in models 10.8 and 11.0).

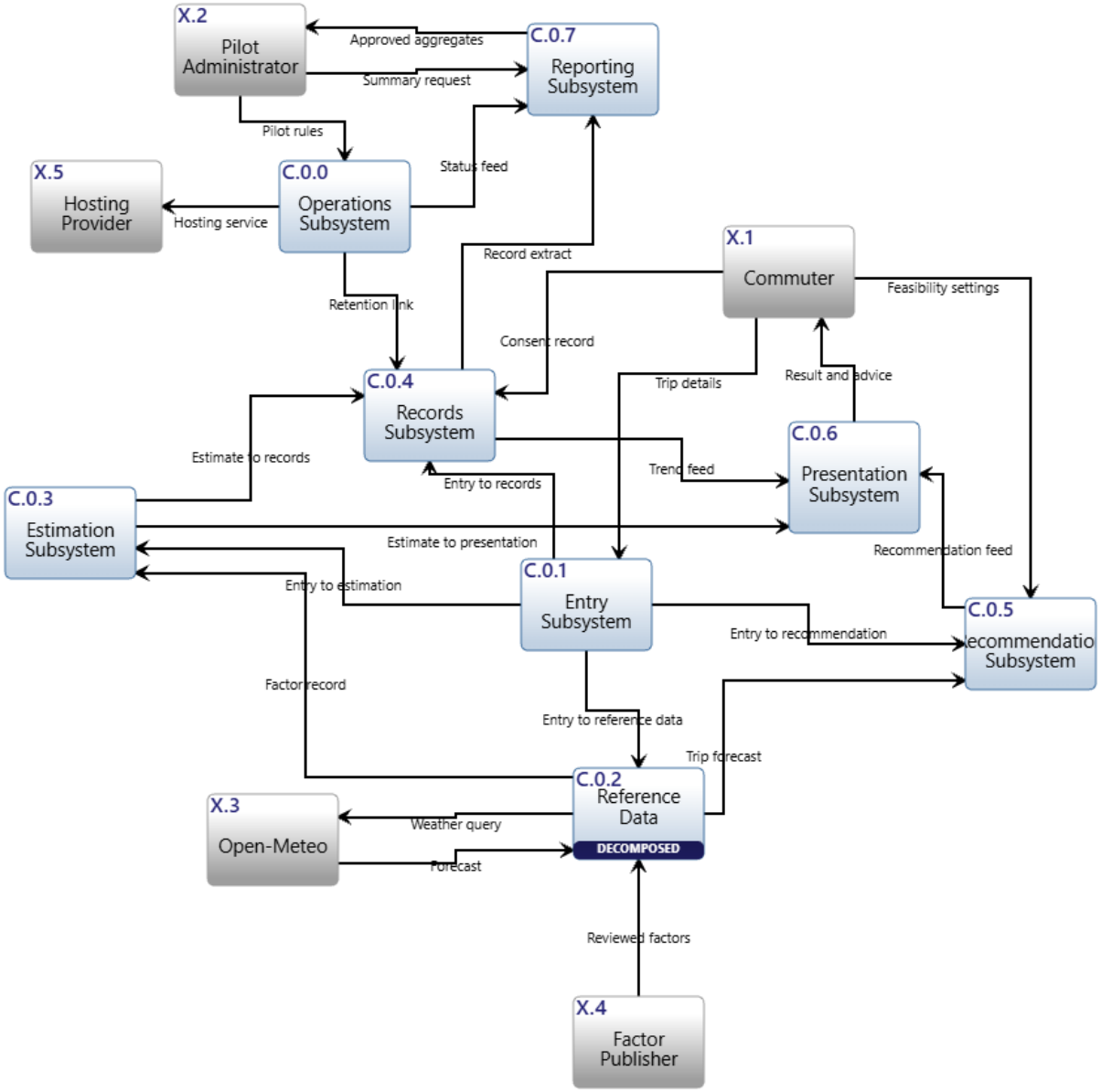


Figure 14. First-level asset diagram: eight subsystems, five external assets and 24 conduits; arrows show the direction of the conduit, and each conduit with its transferred input/outputs is listed in the conduit table.

Table 18. Conduits (interfaces) of the first-level asset diagram (K.1 to K.24) and of C.0.2 (K.2.1, K.2.2). Last column: IO numbers transferred.

Id	Conduit	Connects	Medium	IO
K.1	Trip details	X.1 Commuter ↔ C.0.1 Entry Subsystem	responsive web entry form over HTTPS	1, 2, 3
K.2	Result and advice	C.0.6 Presentation Subsystem ↔ X.1 Commuter	responsive web result view over HTTPS	3
K.3	Consent record	X.1 Commuter ↔ C.0.4 Records Subsystem	consent and account settings page over HTTPS	2
K.4	Feasibility settings	X.1 Commuter ↔ C.0.5 Recommendation Subsystem	feasibility settings page over HTTPS	2
K.5	Weather query	C.0.2 Reference Data Subsystem ↔ X.3 Open-Meteo	HTTPS API request	4
K.6	Forecast	X.3 Open-Meteo ↔ C.0.2 Reference Data Subsystem	structured HTTPS API response	5
K.7	Reviewed factors	X.4 Emission Factor Publisher ↔ C.0.2 Reference Data Subsystem	reviewed file or source-document ingestion	6
K.8	Summary request	X.2 Pilot Administrator ↔ C.0.7 Reporting Subsystem	authenticated dashboard	7, 9
K.9	Approved aggregates	C.0.7 Reporting Subsystem ↔ X.2 Pilot Administrator	authenticated dashboard and approved aggregate export	8
K.10	Pilot rules	X.2 Pilot Administrator ↔ C.0.0 Operations Subsystem	authenticated dashboard	9
K.11	Hosting service	C.0.0 Operations Subsystem ↔ X.5 Hosting Provider	provider deployment and status interface	10, 11
K.12	Entry to reference data	C.0.1 Entry Subsystem ↔ C.0.2 Reference Data Subsystem	internal service API	12
K.13	Entry to estimation	C.0.1 Entry Subsystem ↔ C.0.3 Estimation Subsystem	internal service API	12
K.14	Entry to records	C.0.1 Entry Subsystem ↔ C.0.4 Records Subsystem	internal service API	12, 13
K.15	Entry to recommendation	C.0.1 Entry Subsystem ↔ C.0.5 Recommendation Subsystem	internal service API	12
K.16	Factor record	C.0.2 Reference Data Subsystem ↔ C.0.3 Estimation Subsystem	internal service API	14
K.17	Trip forecast	C.0.2 Reference Data Subsystem ↔ C.0.5 Recommendation Subsystem	internal service API	15
K.18	Estimate to records	C.0.3 Estimation Subsystem ↔ C.0.4 Records Subsystem	internal service API	16
K.19	Estimate to presentation	C.0.3 Estimation Subsystem ↔ C.0.6 Presentation Subsystem	internal service API	16, 17
K.20	Trend feed	C.0.4 Records Subsystem ↔ C.0.6 Presentation Subsystem	internal service API	18
K.21	Record extract	C.0.4 Records Subsystem ↔ C.0.7 Reporting Subsystem	read-only query interface	19
K.22	Recommendation feed	C.0.5 Recommendation Subsystem ↔ C.0.6 Presentation Subsystem	internal service API	20
K.23	Retention link	C.0.0 Operations Subsystem ↔ C.0.4 Records Subsystem	scheduled retention job interface	21
K.24	Status feed	C.0.0 Operations Subsystem ↔ C.0.7 Reporting Subsystem	service log feed	22
K.2.1	Factor set access	C.0.2.1 Factor Import Module ↔ C.0.2.2 Factor Lookup Module	read access to the versioned factor store	23
K.2.2	Query context	C.0.2.3 Weather Client Module ↔ C.0.2.4 Forecast Resolver Module	in-process query handoff	24

Figure 15 is the I2 (interface-square) diagram of the first level. The subsystems sit on the diagonal; the cell in a row and column holds the conduit that carries information from the row's subsystem to the column's subsystem and the input/outputs it transfers; the top row holds the interfaces from external assets and the right-hand column those to external assets; a conduit that carries flows both ways (K.1, K.11) appears once, under the asset where its flow originates. The diagram makes three things visible. First, the interfaces are sparse: 13 of the 56 possible subsystem pairs are used, so coupling is low. Second, C.0.1 Entry is a hub (K.12 to K.15 carry the validated entry to four subsystems) and C.0.4 Records receives from three subsystems, so these two are the places where interface changes would ripple. Third, no subsystem pair interacts in both directions, so there are no cycles. Figure 16 is the I2 diagram of the four modules of C.0.2: only two internal conduits exist (K.2.1 and K.2.2). The conduits that cross the boundary of C.0.2 (K.5, K.6, K.7, K.12, K.16, K.17) connect to C.0.2 as a whole in the model; their allocation to the module that owns each one is given by the sequence diagram (Figure 5) and by the interface requirements 5.3.1 to 5.3.8.

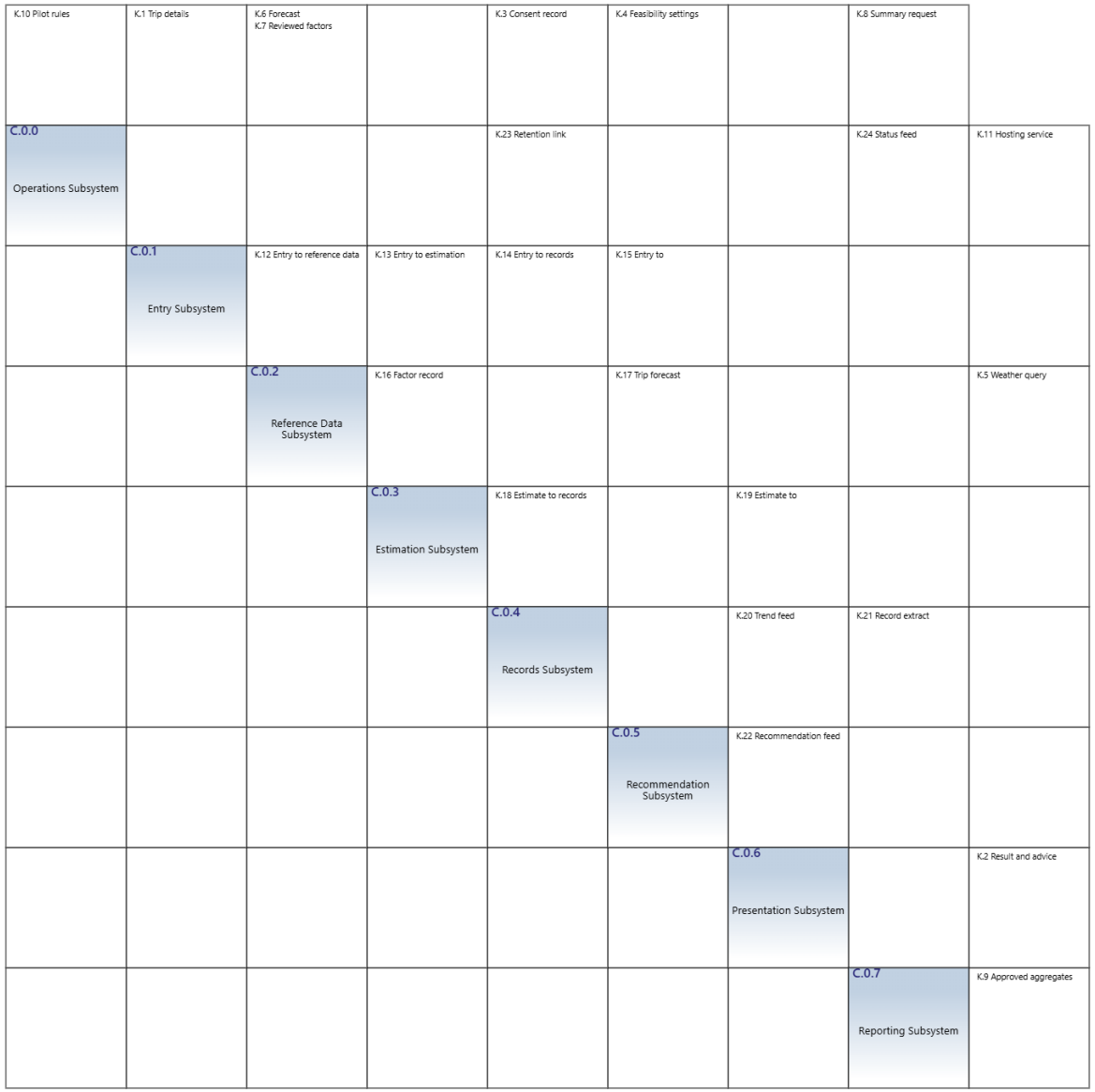


Figure 15. I2 diagram of the first-level assets (model 11.3).

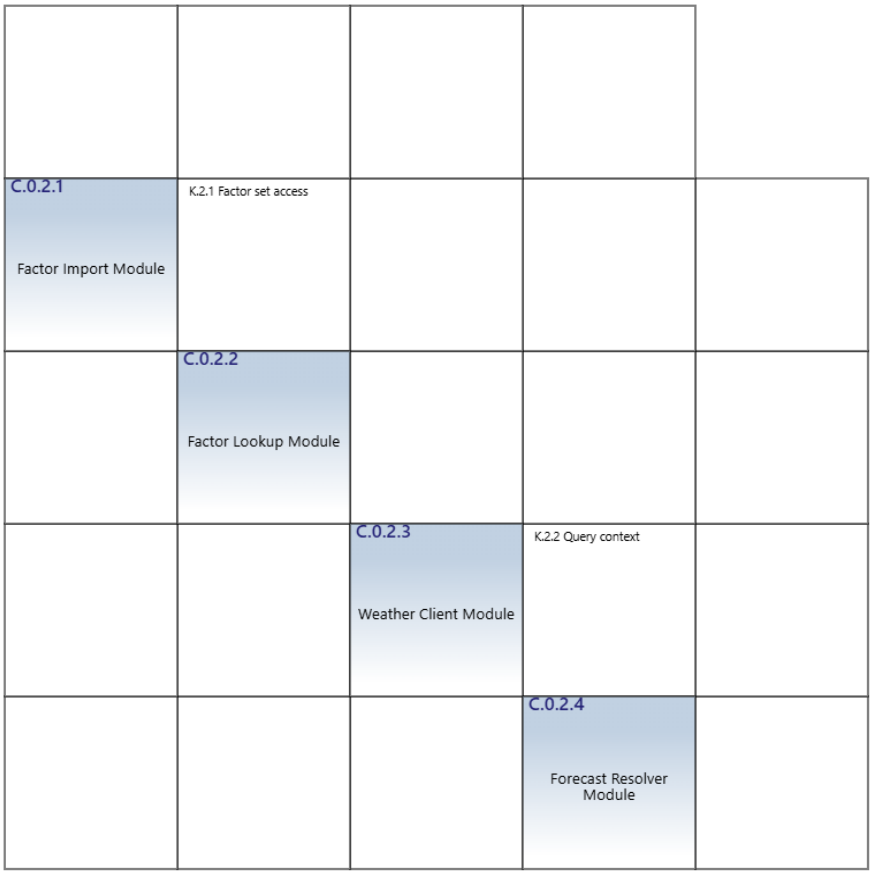


Figure 16. I2 diagram of the second-level assets of C.0.2 (model 11.3).

2.3 Architecture Evaluation

2.3.1 Analysis against Functional Architecture

The physical architecture is checked against the functional architecture in six ways (Table 19). The checks were run by script on the model data so they can be repeated.

Table 19. Analysis of the physical architecture against the functional architecture.

Check	What was compared	Result
One performer per function	Each of F.2.0 to F.2.7 and F.2.2.1 to F.2.2.4 has exactly one performed-by asset; each asset performs exactly one function.	Met (12 of 12 functions)
Hierarchy mirrors function	C.0 is decomposed by 8 subsystems as F.2 is by 8 functions; C.0.2 by 4 modules as F.2.2 by 4 functions; no other asset is decomposed.	Met
Interfaces carry the flows	Every first-level flow, from a source function (or external asset) to each destination, has a conduit between the performing assets that transfers it; no conduit transfers an input/output absent from the IDEF0.	Met (22 of 22 input/outputs; 24 conduits)
Second level closes the first	All six flows of F.2.2 enter or leave a second-level function of F.2.2; IO.23 and IO.24 stay inside.	Met (6 of 6)
Sequence runs on assets	The UC.1 sequence (Figures of Section 2.1.3.2) is continuous over the lifelines C.0.1 to C.0.6 and the external assets, with no interaction lacking a conduit.	Met (14 of 14; 8 of 8)
No tunnels	No tunnel on the Metafunction, the first-level or the second-level IDEF0; no unconnected perimeter arrow.	Met

The analysis also exposed three observations that do not break the criteria but should shape detailed design. (1) IO.2 Commuter Consent and Settings enters three functions from the environment (F.2.1, F.2.4, F.2.5), which

creates three Commuter-facing conduits (K.1, K.3, K.4). Physically the settings page belongs to the Entry Subsystem, which would pass the values on; the functional architecture was baselined in Chapter 7, so the model keeps three conduits and flags the refinement for detailed design. (2) The two interface hubs identified in Section 2.2.1.4 (C.0.1 Entry and C.0.4 Records) should have their interface definitions frozen first. (3) The Presentation Subsystem receives from three subsystems (C.0.3, C.0.4 and C.0.5), which is expected for an assembly function but makes its timing budget the longest dependent chain (Section 3.2.2).

2.4 Alternative Physical Architectures

2.4.1 Evaluation of Physical Alternatives

Three first-level physical architectures were considered. Alternative A is the one-to-one functional architecture of this section. Alternative B keeps the four modules of the Chapter 2 walking skeleton (C.1 to C.4) as the first-level assets. Alternative C is a conventional three-tier layering (web tier, service tier, data tier) with separate operations and reporting assets. Table 20 lists the criteria with their weights and the reason for each; Table 21 scores the alternatives from 1 (poor) to 5 (good).

Table 20. Evaluation criteria and weights (sum 100).

Criterion	Weight	Why it matters
Function-to-form fit	25	Chapter 8 rule: every first-level function is performed by exactly one asset and the physical hierarchy mirrors the functional one.
Requirement allocation	20	Each system requirement can be allocated to and verified against one named asset (3.3.1.x and 3.4.x).
Interface simplicity	15	Fewer and clearer conduits to specify and test; lower coupling.
Privacy and access separation	15	Trip records and aggregates sit in assets that can be access-controlled apart (3.4.5.1 to 3.4.5.3).
Cost and schedule control	15	A budget and a due date can be apportioned to each asset (3.4.1.1, 3.4.1.2).
Reuse of walking skeleton	10	Share of the Chapter 2 code and tests that the structure keeps.

Table 21. Scores of the alternatives. A: functional subsystems; B: walking-skeleton modules; C: three-tier layers.

Criterion	Weight	A	B	C
Function-to-form fit	25	5	2	3
Requirement allocation	20	5	2	3
Interface simplicity	15	3	5	4
Privacy and access separation	15	5	2	3
Cost and schedule control	15	4	2	3
Reuse of walking skeleton	10	3	5	4
Weighted score	100	**4.35**	**2.75**	**3.25**

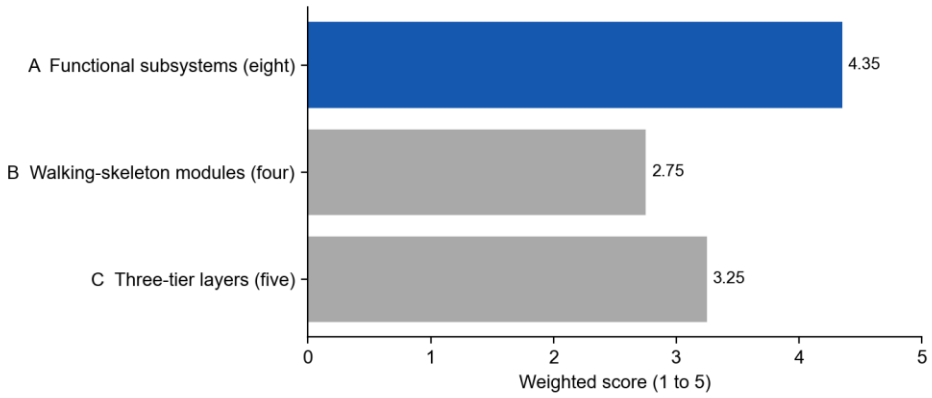


Figure 17. Weighted scores of the three physical alternatives.

Alternative A, Functional subsystems (eight). One subsystem per first-level function: C.0.0 to C.0.7. Matches F.2.0 to F.2.7 one to one. Costs the most interfaces (24 conduits) and needs a thin adapter to the existing code layout.

Alternative B, Walking-skeleton modules (four). C.1 Web Front End, C.2 Application Service, C.3 Data Adapter, C.4 Explanation Model under C.0. Few interfaces, but C.1 and C.4 each serve two functions, C.2 and C.3 cross F.2.3 to F.2.5 and F.2.2 to F.2.4, and F.2.0 and F.2.7 have no asset. It cannot satisfy the one-to-one rule or give a clean budget per asset.

Alternative C, Three-tier layers (five). Web tier, service tier, data tier, plus operations and reporting assets. Layers group by technology, not function: the service tier performs six functions, so requirement allocation and cost apportionment stay coarse. Operations and reporting still need their own assets.

Alternative A scores 4.35, against 2.75 for B and 3.25 for C, and is the only one that meets the Chapter 8 one-to-one rule. It is selected. Its weaknesses are the interface count and the gap to the existing code; the first is a cost the lab accepts for traceability, the second is closed by Table 13. The scores are team judgements. A leads by more than one point, and shifting 10 points of weight between any two criteria changes a weighted score by at most 0.4, so the ranking holds.

3 System Requirements Management

3.1 Requirements Traceability

Chapter 8 also completes the system-level specification. Document 3 of the model was generated from the Metafunction and first-level IDEF0 (sections 3.1 to 3.3, with the input, output, interface and functional requirements) and edited; the non-functional requirements of section 3.4 were added by hand and traced to the system asset. Table 22 gives the leaf requirement count of each section after the update. The 17 leaf requirements of 3.4 are satisfied by C.0, and every functional requirement 3.3.1.k is satisfied by the function F.2.k that performs it and, through the allocation, by its subsystem.

Table 22. System requirements document (3) after the update: leaf requirements per section.

Section	Content	Leaf requirements
3.1.1	Input requirements (accept ... from ...)	14
3.1.2	Output requirements (provide ... to ...)	9
3.2	External interface requirements (utilize ... to interface with ...)	11
3.3	Functional requirements (one per function)	17
3.4	System-wide requirements: cost and schedule, performance, availability and capacity, accessibility, privacy and security, estimation quality	17

Figure 18 is the spider diagram of the Reference Data Subsystem, C.0.2, from model 11.3. At the centre is the asset; around it are the functions it performs (F.2.2 and its four sub-functions), the modules that decompose it, the conduits that cross or lie inside it, the system requirements the component requirements are traced from, and the five groups of component requirements it satisfies. The same relationships exist for every first-level subsystem: each leaf function has a performed-by asset, a satisfied-by design output requirement (documents 4 and 5 of the model) and a traced-from system requirement 3.3.1.x.

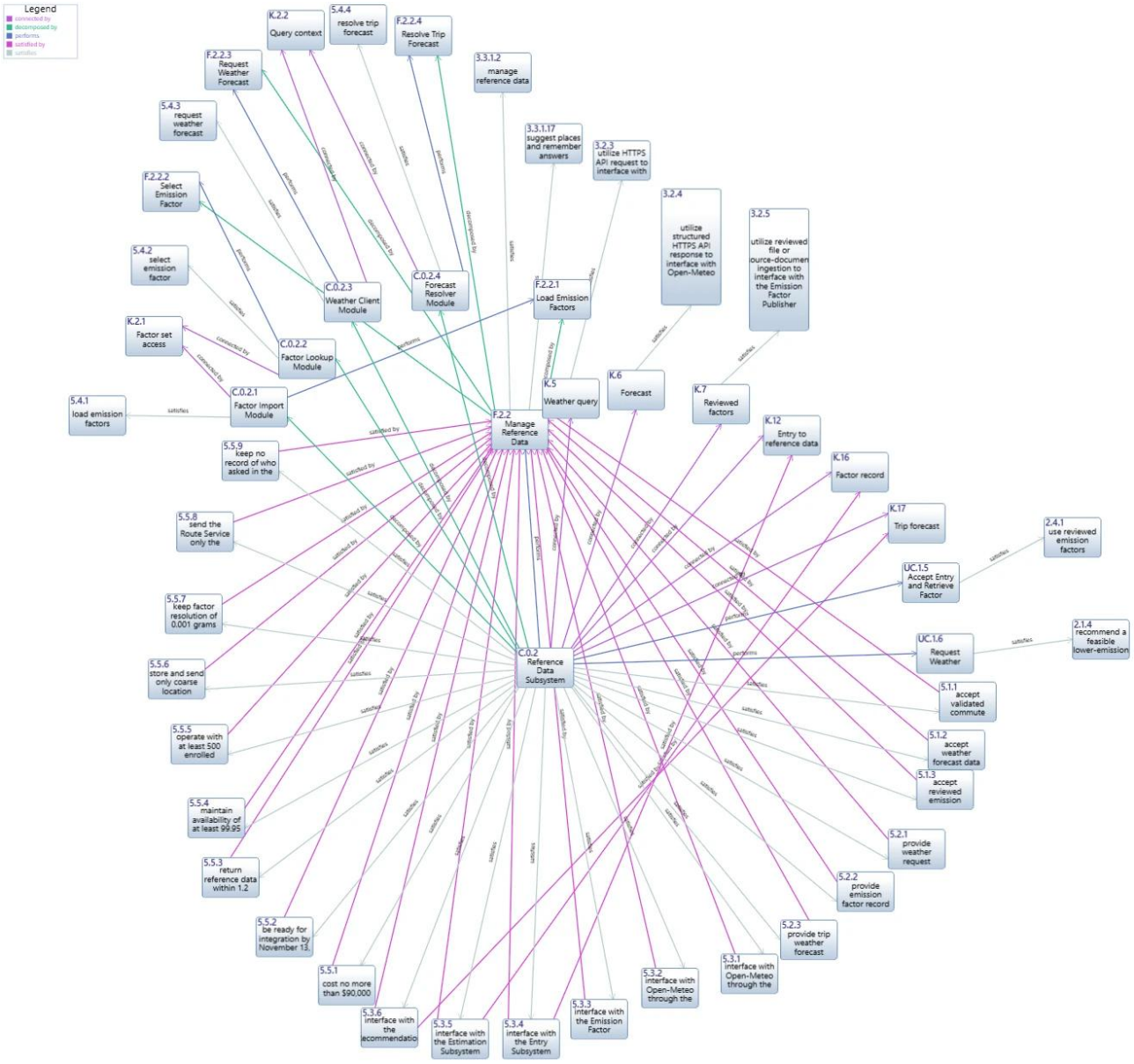


Figure 18. Spider diagram of C.0.2 Reference Data Subsystem: functions, modules, conduits, system requirements and component requirements.

Table 23. Function, asset, design output requirement, system requirement and stakeholder requirements for every first-level function.

Function	Asset	Design output requirement	System requirement	Stakeholder requirements
F.2.0	C.0.0 Operations Subsystem	4.1.1	3.3.1.8	2.2.4, 2.3.3, 2.3.6, 2.3.7
F.2.1	C.0.1 Entry Subsystem	4.1.2	3.3.1.1	2.1.1, 2.1.8, 2.1.9, 2.5.2
F.2.3	C.0.3 Estimation Subsystem	4.1.3	3.3.1.3	2.1.2, 2.1.3, 2.4.3
F.2.4	C.0.4 Records Subsystem	4.1.4	3.3.1.4	2.1.6, 2.1.7, 2.1.8, 2.3.2, 2.3.6
F.2.5	C.0.5 Recommendation Subsystem	4.1.5	3.3.1.5	2.1.4, 2.1.5
F.2.6	C.0.6 Presentation Subsystem	4.1.6	3.3.1.6	2.1.2, 2.1.3, 2.1.6, 2.1.9, 2.1.10
F.2.7	C.0.7 Reporting Subsystem	4.1.7	3.3.1.7	2.2.1, 2.2.2, 2.2.3, 2.3.4, 2.3.7
F.2.2	C.0.2 Reference Data Subsystem	5.4.1 to 5.4.4	3.3.1.2	2.4.1, 2.1.4, 2.1.3, 2.5.1

Models 10.9 and 11.0 added the trip planner (UC.3) and accounts (UC.4) and nine functional requirements, 3.3.1.9 to 3.3.1.17. They were allocated to the existing first-level functions without adding a function, so each lands on an existing subsystem (Table 24): the form architecture absorbs the new features without a new asset. Two consequences follow. The Records Subsystem takes five of the nine, which confirms it as an interface and load hub (Section 2.3.1). And 3.3.1.17 (place suggestion and the persistent place cache) enlarges F.2.2 beyond the four second-level functions of this section; a fifth function, Suggest Places, with a module C.0.2.5, is the planned next step, and the two system-wide requirements that touch it are already flowed down (5.5.8 and 5.5.9). The model keeps the walking-skeleton modules C.1 to C.4 as performers of the UC.3 and UC.4 actions, because the code tests read them; they are a code view of the subsystems (Table 13).

Table 24. Functional requirements added in models 10.9 and 11.0, the first-level function they are allocated to, and the subsystem that performs it.

Requirement	Name	Function	Subsystem
3.3.1.9	compare travel modes	F.2.3, F.2.5	C.0.3 Estimation, C.0.5 Recommendation
3.3.1.10	confirm completed trips	F.2.4	C.0.4 Records
3.3.1.11	evaluate achievements	F.2.4	C.0.4 Records
3.3.1.12	track campus progress	F.2.4, F.2.6	C.0.4 Records, C.0.6 Presentation
3.3.1.13	plan direct bus rides	F.2.5	C.0.5 Recommendation
3.3.1.14	share a car trip's emissions among its occupants	F.2.3	C.0.3 Estimation
3.3.1.15	manage accounts and invite codes	F.2.4	C.0.4 Records
3.3.1.16	keep trips per account	F.2.4	C.0.4 Records
3.3.1.17	suggest places and remember answers	F.2.2	C.0.2 Reference Data

The model also keeps the earlier traces: the 17 system-wide requirements are satisfied by C.0; the functional requirements 3.3.1.k remain satisfied by the functions F.2.k; the external interface requirements 3.2.1 to 3.2.8 remain satisfied by C.0 and the external assets and are now also satisfied by the conduits that realise them.

3.2 System Design Output Requirements

3.2.1 Methodology for Deriving System Design Output Requirements

A system design output requirement (SDOR) states what a form entity must do, in the form "The [named subsystem] shall ...", so that its designer can design and test to it. The team derived them in five steps.

- **Select the leaf functions.** A leaf function is a function that is not decomposed: F.2.0, F.2.1 and F.2.3 to F.2.7 at the first level and F.2.2.1 to F.2.2.4 at the second level (11 in all).
- **Find the performer.** Use the one-to-one allocation of Section 2.2.1.1; the subject of the requirement is that asset.

- **State the requirement.** One "shall" per requirement, with the function as its predicate. For leaf first-level functions, the statement restates the system functional requirement 3.3.1.k for the subsystem and adds the specific obligation that makes it testable (validate, restrict, suppress, restate only calculated facts).
- **Map, justify and measure.** Each requirement names its higher-level requirement (the system requirement 3.3.1.k, or for second-level functions 3.3.1.2), its stakeholder requirements, its rationale, a measure of performance (MOP) with a target or threshold and verification criteria with a method (Test, Inspection, Analysis, Demonstration). Targets are those of the stakeholder requirements.
- **Complete the component specification.** For the component chosen in Section 2.1.1.3, add the input, output, interface and system-wide requirements of Chapter 9. Input and output requirements come from the first-level flows of F.2.2 (internal second-level flows IO.23 and IO.24 stay out because they do not cross the component boundary); interface requirements come from the conduits that cross the boundary; system-wide requirements come from 3.4 by equivalence, apportionment or synthesis.

The three flow-down methods are used as follows. **Equivalence** applies the same requirement to the component when it cannot be shared out (capacity, privacy limits). **Apportionment** divides a quantity that adds up across the system (cost, time, unavailability) so that the shares total 100% of the system value including a reserve. **Synthesis** derives a component value from a system value through an engineering relationship (here, factor resolution from a rounding tolerance and a longest trip). The method is stated for every system-wide requirement with the reason it was chosen.

3.2.2 Detailed System Design Output Requirements

3.2.2.1 Subsystem Design Output Requirements

The seven leaf first-level functions have one design output requirement each, in document 4 of the model. They are presented in the summary table of Section 3.2.3 together with the second-level requirements. All use the form "The [named] Subsystem shall ...", map to a system functional requirement, and carry rationale, MOP and verification criteria.

3.2.2.2 Component Specification for the Reference Data Subsystem

The component specification (document 5 of the model) is kept apart from the system requirements, as the lab requires. It contains 27 requirements: three input, three output, eight interface, four functional (one for each of F.2.2.1 to F.2.2.4, so every function of the second-level IDEF0 has exactly one requirement) and nine system-wide. Table 25 gives the input, output and interface requirements; the functional requirements are in Table 32; and the system-wide requirements are in Table 26. Every input, control and output that enters or leaves the second-level IDEF0 has a requirement: IO.5, IO.6 and IO.12 as inputs (5.1.1 to 5.1.3) and IO.4, IO.14 and IO.15 as outputs (5.2.1 to 5.2.3). An interface requirement exists for each external asset the component touches: Open-Meteo through the Weather query and Forecast conduits (5.3.1, 5.3.2), the Emission Factor Publisher through Reviewed factors (5.3.3) and, since model 11.0 allocated place suggestion to F.2.2, the Route Service through Route request and Route response (5.3.7, 5.3.8); and for each neighbouring subsystem (Entry, Estimation, Recommendation; 5.3.4 to 5.3.6).

Table 25. Component specification, input, output and interface requirements for C.0.2 (document 5, sections 5.1 to 5.3).

ID	Requirement	Parent	Rationale	MOP	Verification criteria (method)
5.1.1	The Reference Data Subsystem shall accept validated commute entries from the Entry Subsystem.	3.3.1.2, 4.1.2	The factor and the forecast are chosen per trip, so the subsystem needs the mode, geography, coarse area and trip time of every accepted entry.	Share of validated entries accepted without loss of mode, geography, coarse area or trip time (target 100%).	Replay a set of validated entries and compare the fields received with the fields sent. (Test)
5.1.2	The Reference Data Subsystem shall accept weather forecast data, with a forecast timestamp, from Open-Meteo.	3.1.1.3	The recommendation rule needs forecast weather and the time the forecast was made so that a stale forecast can be recognised.	Share of well-formed forecast responses accepted (target 100%); malformed responses that stop the subsystem (target 0).	Feed fixtures of valid, empty and malformed Open-Meteo responses through the Weather Client interface. (Test)
5.1.3	The Reference Data Subsystem shall accept reviewed emission factors, with source, units, geography, methodology and version, from the Emission Factor Publisher.	3.1.1.4	Only reviewed, versioned factors may govern an estimate; the five attributes are what the disclosure to the Commuter is built from.	Share of accepted factor sets that carry all five attributes (target 100%); sets accepted with a missing attribute (target 0).	Load a valid factor file and files each missing one attribute; inspect the store. (Test)
5.2.1	The Reference Data Subsystem shall provide weather requests, containing only a coarse location and the trip time, to Open-Meteo.	3.1.2.2	Open-Meteo is a third party; nothing finer than a coarse area may leave the system (need 1.5.1).	Share of outgoing weather requests that contain only a coarse area and a trip time (target 100%).	Capture every request sent in a test run and inspect its fields. (Inspection)
5.2.2	The Reference Data Subsystem shall provide the applicable emission factor record, with source, units, geography and version, to the Estimation Subsystem.	3.3.1.2, 4.1.3	The Estimation Subsystem must show where each number comes from, so the provenance travels with the factor.	Share of emission factor records provided with source, units, geography and version (target 100%).	Inspect the records returned for a sample of 50 reference entries. (Inspection)
5.2.3	The Reference Data Subsystem shall provide the trip weather forecast, with its forecast timestamp or a forecast-unavailable flag, to the Recommendation Subsystem.	3.3.1.2, 4.1.5	The recommendation rule must know whether it can rely on the forecast; an explicit flag avoids silent assumptions.	Share of trip forecasts provided with a timestamp or an unavailable flag (target 100%).	Run entries with a live stub, a late stub and no response; inspect each output. (Test)
5.3.1	The Reference Data Subsystem shall interface with Open-Meteo through the Weather query conduit using an HTTPS API request.	3.2.3	Carries the external interface requirement 3.2.3 down to the subsystem that owns the call.	Share of weather queries sent over HTTPS to the documented endpoint (target 100%).	Inspect the client configuration and the traffic of a test run. (Inspection)
5.3.2	The Reference Data Subsystem shall interface with Open-Meteo through the Forecast conduit using a structured HTTPS API response.	3.2.4	Carries the external interface requirement 3.2.4 down; a structured response can be validated against a schema.	Share of responses validated against the response schema before use (target 100%).	Feed schema-conforming and non-conforming responses; confirm only the former are used. (Test)
5.3.3	The Reference Data Subsystem shall interface with the Emission Factor Publisher through the Reviewed factors conduit using reviewed file or source-document ingestion.	3.2.5	Carries the external interface requirement 3.2.5 down; factors come from a reviewed file, not a live call.	Share of factor sets loaded from a reviewed file or source document (target 100%).	Inspect the load log of the pilot build for the source of every factor set. (Inspection)
5.3.4	The Reference Data Subsystem shall interface with the Entry Subsystem through the Entry to reference data conduit using an internal service API.	3.3.1.2, 4.1.2	Internal conduit K.12 of the first-level asset diagram; it carries IO.12 Validated Commute Entry.	Share of calls on the conduit that follow the interface definition (target 100%).	Contract test of the internal service API. (Test)
5.3.5	The Reference Data Subsystem shall interface with the Estimation Subsystem through the Factor record conduit using an internal service API.	3.3.1.2, 4.1.3	Internal conduit K.16; it carries IO.14 Emission Factor Record.	Share of calls on the conduit that follow the interface definition (target 100%).	Contract test of the internal service API. (Test)
5.3.6	The Reference Data Subsystem shall interface with the Recommendation Subsystem through the Trip forecast conduit using an internal service API.	3.3.1.2, 4.1.5	Internal conduit K.17; it carries IO.15 Trip Weather Forecast.	Share of calls on the conduit that follow the interface definition (target 100%).	Contract test of the internal service API. (Test)
5.3.7	The Reference Data Subsystem shall interface with the Route Service through the Route request conduit using an HTTPS request.	3.2.9	Added by model 11.0: place suggestion and place resolution (system requirement 3.3.1.17) are allocated to F.2.2 and call the Route Service. This interaction is not yet in the second-level IDEF0, which gets a fifth function in the next iteration.	Share of Route Service calls sent over HTTPS to the documented endpoint (target 100%).	Inspect the route adapter configuration and the traffic of a test run. (Inspection)
5.3.8	The Reference Data Subsystem shall interface with the Route Service through the Route response conduit using a structured HTTPS response.	3.2.10	Added by model 11.0: carries the external interface requirement 3.2.10 down; a structured response can be validated before use and replaced by a labelled fallback when it is malformed.	Share of Route Service responses validated before use (target 100%); malformed responses that stop the subsystem (target 0).	Feed conforming, malformed and missing responses through the route adapter. (Test)

3.2.2.3 System-Wide Requirements Flow-Down

Table 26 lists the nine system-wide requirements flowed down to C.0.2 and the method for each, with the reason. Eight requirements of 3.4 were left out because the component does not influence them (Table 28). The three apportionment budgets follow: cost (Table 29), response time (Table 30) and unavailability (Table 31). Each budget totals the system value, so the sum of the subsystem requirements guarantees the system requirement when the subsystems are integrated.

Table 26. System-wide requirements of the Reference Data Subsystem with flow-down method and rationale (document 5, section 5.5).

ID	Requirement	System req.	Method and rationale	MOP	Verification criteria (method)
5.5.1	The Reference Data Subsystem shall cost no more than $90,000 to develop and deploy.	3.4.1.1	**Apportionment**. Apportionment of the $750,000 system budget: 12% to this subsystem (budget in the cost apportionment table of Section 3.2.2). The share reflects two external interfaces and a versioned store; a detailed estimate is deferred to the design stage.	Cost variance against $90,000 (target 0% or less).	Subsystem cost forecast reviewed at each stage gate. (Analysis)
5.5.2	The Reference Data Subsystem shall pass its verification tests and be ready for system integration no later than November 13, 2026.	3.4.1.2	**Apportionment**. Apportionment of the schedule: results are due December 18, 2026 and the pilot needs four weeks of use, so the pilot must start by November 16, 2026 and the subsystems must be integrated before that (milestone table in Section 3.2.2). The Estimation Subsystem depends on the factor store, so this subsystem verifies first, two days before the other subsystems. The web application is already implemented, so the remaining work is verification and integration.	Date on which all subsystem verification tests pass (threshold November 13, 2026 for integration; November 10, 2026 for this subsystem).	Schedule review at the integration gate. (Analysis)
5.5.3	The Reference Data Subsystem shall return the emission factor record and the trip weather forecast within 1.2 seconds of receiving a validated commute entry, using no more than 1.0 second for the Open-Meteo call.	3.4.2.1	**Apportionment**. Apportionment of the 3-second result time across the path (budget in the response-time table of Section 3.2.2). The external call dominates this subsystem's share, so the call is capped at 1.0 second and the rest at 0.2 second.	95th percentile time from receipt of the entry to both outputs (threshold 1.2 seconds).	Timing test with an Open-Meteo stub at 1.0 second latency and with the live service on three reference devices. (Test)
5.5.4	The Reference Data Subsystem shall maintain an availability of at least 99.95 percent during each month of the pilot, excluding Open-Meteo outages.	3.4.3.1	**Apportionment**. Apportionment of the 0.5% monthly unavailability allowed by the system requirement (budget in the availability table of Section 3.2.2): 0.05% to this subsystem. Open-Meteo outages are outside the budget because requirement 5.4.4 keeps the estimate running without a forecast.	Measured monthly availability of the subsystem (threshold 99.95%).	Monitor a successful service check every minute and report monthly. (Analysis)
5.5.5	The Reference Data Subsystem shall operate without functional failure with at least 500 enrolled Commuter accounts.	3.4.3.2	**Equivalence**. Equivalence: capacity cannot be divided among subsystems, because each trip on the path passes through every one of them. The same value applies to this subsystem.	Number of enrolled accounts with which the subsystem operates without functional failure (target 500 or more).	Run 500 test accounts through the entry task and confirm no functional failure. (Test)
5.5.6	The Reference Data Subsystem shall limit every location that it stores or sends to Open-Meteo to coarse-area granularity.	3.4.5.2	**Equivalence**. Equivalence: the privacy limit is not a quantity to share out; every subsystem that handles a location must meet it in full.	Number of stored or transmitted locations finer than a coarse area (target 0).	Network and storage inspection of the pilot build. (Inspection)
5.5.7	The Reference Data Subsystem shall store and return each emission factor with a resolution of at least 0.001 grams of CO2e per kilometre.	3.4.6.1	**Synthesis**. Synthesis: the system must reproduce each reference result to the nearest gram, an error of at most 0.5 g. For a trip of up to 100 km, a factor rounded to 0.001 g/km changes the result by at most 100 km x 0.0005 g/km = 0.05 g, one tenth of that tolerance. The resolution is derived from the tolerance and the longest trip.	Resolution of stored and returned factors (threshold 0.001 g CO2e per km).	Compare stored factors with the published values; run the reference cases for trips up to 100 km. (Test)
5.5.8	The Reference Data Subsystem shall send the Route Service only the place text the Commuter entered and the coordinates resolved from it.	3.4.5.4	**Equivalence**. Equivalence: since model 11.0 the subsystem also resolves and suggests places (system requirement 3.3.1.17 is allocated to F.2.2), so it is the asset that calls the Route Service; the limit is a privacy constraint that cannot be divided and applies in full.	Number of Route Service requests that carry data other than the entered place text and resolved coordinates (target 0).	Capture the requests of a test run and inspect their fields. (Inspection)
5.5.9	The Reference Data Subsystem shall keep cached place and route answers without any record of which account asked.	3.4.5.7	**Equivalence**. Equivalence: the persistent place cache of model 11.0 lives in this subsystem; separating answers from accounts is a privacy limit that every holder of the cache must meet in full.	Number of cache entries that identify the requesting account (target 0).	Inspect the cache store after a multi-account test run. (Inspection)

Table 27 gives the schedule and where the project stands on October 30, 2026. The web application is already implemented (the use cases UC.1, UC.3 and UC.4 run, with an automated test suite), so the project is at the end of subsystem implementation and at the start of verification and integration. Two dates are fixed: system verification ends on December 2, 2026 and the sponsor's results are due on December 18, 2026. Working back from them, the integration date of requirement 5.5.2 is November 13, 2026 and the pilot starts on November 16, which allows four weeks of use. The earlier assumption of eight weeks does not fit, so the weekly-active-use measure of stakeholder requirement 2.1.6 is set to week four in model 11.3 (it read week eight); the sponsor should confirm this change, and the 40% target was not changed. Status is as reported by the team.

Table 27. Schedule milestones, with status on October 30, 2026.

Date	Milestone	Owner or note	Status
2026-10-30	Requirements baselined and model 11.0 imported; the web application implements UC.1, UC.3 and UC.4 with an automated test suite	All subsystems	Done
2026-11-06	Interfaces K.1 to K.24 frozen and written into docs/deployment.md; documents 4 and 5 reviewed	C.0.1, C.0.4, all	In progress
2026-11-10	Reference Data Subsystem passes the tests derived from its component specification	C.0.2; Estimation depends on it	Planned
2026-11-12	All other subsystems pass the tests derived from their design output requirements	C.0.0, C.0.1, C.0.3 to C.0.7	Planned
2026-11-13	**System integration date**: eight subsystems integrated, invited pilot accounts created (requirement 5.5.2)	C.0 Integration	Planned
2026-11-16	Pilot starts; four weeks of use before data collection closes	C.0.0 Operations	Planned
2026-12-02	System verification complete (response time, availability, access and suppression tests)	C.0 Test	Planned
2026-12-11	Pilot data collection closes; evaluation and report written	C.0.7 Reporting	Planned
2026-12-18	Pilot results due to the Company Sponsor (requirement 3.4.1.2)	C.0.7 Reporting	Planned

Table 28. System-wide requirements not flowed down to C.0.2, with the reason.

System requirement	Reason it does not apply
3.4.2.2 complete commute entry in 30 seconds	Entry time is spent on the entry form; this subsystem has no user interface and is off the Commuter's path while the form is filled in.
3.4.4.1 conform to WCAG 2.2 Level AA	Applies to the entry and result views; the Reference Data Subsystem has no view.
3.4.5.1 restrict access to individual trip records	The subsystem keeps no trip records; it receives an entry, uses it and does not store it.
3.4.5.3 suppress aggregate groups below 5	Applies where aggregates are made, in the Reporting Subsystem.
3.4.5.5 store passwords and invite codes only as hashes	Accounts are managed by the Records Subsystem (3.3.1.15 is allocated to F.2.4).
3.4.5.6 lock log-in after repeated wrong passwords	Log-in handling belongs to the Records and Entry Subsystems; this subsystem has no log-in.
3.4.6.2 label route-based figures as estimates	Labelling happens where the figure is estimated and shown, in the Estimation and Presentation Subsystems.
3.4.6.3 state that bus times come from a timetable	The timetable is read and shown by the Recommendation and Presentation Subsystems (3.3.1.13 is allocated to F.2.5).

Table 29. Apportionment of the $750,000 cost requirement (3.4.1.1).

Subsystem	Share	Budget
C.0.0 Operations Subsystem	12%	$90,000
C.0.1 Entry Subsystem	12%	$90,000
C.0.2 Reference Data Subsystem	12%	$90,000
C.0.3 Estimation Subsystem	10%	$75,000
C.0.4 Records Subsystem	14%	$105,000
C.0.5 Recommendation Subsystem	8%	$60,000
C.0.6 Presentation Subsystem	14%	$105,000
C.0.7 Reporting Subsystem	8%	$60,000
Integration and test	6%	$45,000
Management reserve	4%	$30,000
Total	**100%**	**$750,000**

Table 30. Apportionment of the 3-second result time (3.4.2.1) along the UC.1 path, summed serially as a conservative assumption.

Subsystem	Time (s)
C.0.1 Entry Subsystem	0.3
C.0.2 Reference Data Subsystem	1.2
C.0.3 Estimation Subsystem	0.2
C.0.4 Records Subsystem	0.3
C.0.5 Recommendation Subsystem	0.2
C.0.6 Presentation Subsystem	0.5
Reserve	0.3
Total	**3.0**

Table 31. Apportionment of the 0.5% monthly unavailability allowed by 3.4.3.1 across the subsystems on the UC.1 path.

Subsystem	Unavailability allowed	Availability required
C.0.0 Operations Subsystem	0.10%	99.90%
C.0.1 Entry Subsystem	0.07%	99.93%
C.0.2 Reference Data Subsystem	0.05%	99.95%
C.0.3 Estimation Subsystem	0.03%	99.97%
C.0.4 Records Subsystem	0.10%	99.90%
C.0.5 Recommendation Subsystem	0.03%	99.97%
C.0.6 Presentation Subsystem	0.07%	99.93%
Reserve	0.05%	
Total	**0.50%**	**99.50%**

3.2.3 Summary Table of System Design Output Requirements

Table 32 summarises the design output requirements of all 11 leaf functions that are allocated to form entities: the seven first-level subsystems and the four functions of the Reference Data Subsystem. Each row has the requirement, the higher-level requirement it maps to, the rationale, the MOP and the verification criteria with the method.

Table 32. Summary of system design output requirements for the leaf functions allocated to form entities (documents 4 and 5).

ID	Asset / function	Requirement	Parent	Rationale	MOP	Verification criteria (method)
4.1.1	C.0.0 Operations Subsystem / F.2.0	The Operations Subsystem shall operate the pilot service by deploying releases to the Hosting Provider, monitoring availability and running retirement.	3.3.1.8	A service that stays up and can be retired cleanly is a pilot condition (needs 1.2.3 and 1.3.4); one asset owns release, monitoring and retirement so accountability is not shared.	Share of one-minute availability checks passed in a month (target 99.5% or more); identifiable records left after a retirement rehearsal (target 0).	Review the monthly monitoring report; run a retirement rehearsal on a copy of the pilot data. (Analysis)
4.1.2	C.0.1 Entry Subsystem / F.2.1	The Entry Subsystem shall collect commute entries through a responsive web entry form, validating each entry and checking recorded consent before the entry is passed on.	3.3.1.1	Entry effort is the main barrier to repeated logging; invalid or unconsented entries must stop at the boundary so no other subsystem handles them.	Share of timed entry tasks completed in 30 seconds or less (target 80% or more); invalid or unconsented entries passed on (target 0).	Pilot usability session with at least 20 participants logging five trips each; rule test set of invalid and unconsented entries. (Test)
4.1.3	C.0.3 Estimation Subsystem / F.2.3	The Estimation Subsystem shall estimate the CO2e of each validated trip from the applicable emission factor and the trip distance, and its difference from the always-drive baseline.	3.3.1.3	The estimate is the core product value; giving the factor source, version and distance convention with each result lets reviewers trace every number (need 1.4.1).	Difference between computed and hand-calculated CO2e for the agreed reference cases (target 0 g after rounding to the nearest gram); share of results carrying source, version and distance convention (target 100%).	Run the reference-case set against the Estimation Subsystem; inspect a sample of 50 results. (Test)
4.1.4	C.0.4 Records Subsystem / F.2.4	The Records Subsystem shall store each accepted trip record, update weekly and monthly trends, restrict access to the owning Commuter and delete records on request and at retirement.	3.3.1.4	Trip records are personal data; keeping storage, trends and deletion in one asset gives one place to enforce consent, access and deletion (needs 1.1.6 and 1.3.4).	Trip records exposed to any other role in access tests (target 0); deletion requests that leave an identifiable record (target 0); accepted entries without exactly one timestamped event (target 0).	Role-based access test matrix; deletion test on test accounts including backups in scope; reconcile the event log with accepted entries. (Test)
4.1.5	C.0.5 Recommendation Subsystem / F.2.5	The Recommendation Subsystem shall select one feasible lower-emission alternative, with a stated reason, from the trip distance, the forecast weather and the Commuter's feasibility settings.	3.3.1.5	Advice changes behaviour only if it fits the Commuter's situation; explicit rules can be inspected and explained (need 1.1.4).	Recommendations that include a mode marked not feasible (target 0); share of eligible trips with one alternative and a reason (target 100%).	Rule test set covering every combination of excluded modes and weather states. (Test)
4.1.6	C.0.6 Presentation Subsystem / F.2.6	The Presentation Subsystem shall present each trip result, with trend, baseline difference, method disclosure, alternative and a plain-language explanation that restates only calculated facts, in a responsive result view.	3.3.1.6	The result view is where the Commuter meets the model output; the explanation may not calculate or invent (need 1.1.8, risk R8).	Time from entry submission to result display (threshold 3 seconds); sampled explanations with a figure, condition or mode not in the calculated result (target 0); unresolved WCAG 2.2 Level AA failures (target 0).	Timing test on three reference devices; inspect 50 explanations field by field against the calculated result; accessibility audit of the result view. (Test)
4.1.7	C.0.7 Reporting Subsystem / F.2.7	The Reporting Subsystem shall provide an authorized Pilot Administrator with approved aggregate summaries that exclude individual identifiers and suppress any group of fewer than 5 Commuters.	3.3.1.7	Institutional evidence must not re-identify anyone; suppression and exclusion are applied where the aggregate is made, not downstream (need 1.2.2).	Reported groups with fewer than 5 Commuters (target 0); individual identifiers or trip records in outputs (target 0); measures of effectiveness reported before December 18, 2026 (target all defined).	Aggregation test with seeded groups of size 1 to 10; output inspection; evaluation report checked against the measure list at the launch-decision gate. (Test)
5.4.1	C.0.2.1 Factor Import Module / F.2.2.1	The Reference Data Subsystem shall load each reviewed emission factor set into a versioned factor set and reject any set that lacks source, units, geography, methodology or version.	3.3.1.2	Unreviewed or unversioned factors would let an estimate that cannot be traced reach a Commuter (risk TR.1).	Share of factor sets in the store that carry all five attributes (target 100%); incomplete sets that enter the store (target 0).	Load valid and incomplete files; inspect the store and the rejection log. (Test)
5.4.2	C.0.2.2 Factor Lookup Module / F.2.2.2	The Reference Data Subsystem shall select the applicable emission factor for each validated commute entry from the versioned factor set and return it with its provenance.	3.3.1.2	The factor depends on the travel mode and the geography; the Commuter must see which version produced the number.	Share of reference entries for which the expected factor and version are returned (target 100%); selection time (threshold 0.2 seconds).	Run a reference table of mode and geography combinations and compare with the expected factor. (Test)
5.4.3	C.0.2.3 Weather Client Module / F.2.2.3	The Reference Data Subsystem shall request a weather forecast from Open-Meteo for the coarse location and the time of each validated commute entry.	3.3.1.2	The recommendation rule depends on forecast weather; the request is built here so that the privacy limit is enforced in one place.	Share of validated entries for which a request is sent (target 100%); requests carrying data finer than coarse area and time (target 0).	Capture requests during a run of reference entries. (Test)
5.4.4	C.0.2.4 Forecast Resolver Module / F.2.2.4	The Reference Data Subsystem shall resolve the forecast returned by Open-Meteo to the forecast for the trip location and time, with its forecast timestamp, or flag the forecast unavailable when no response arrives within 1.0 second.	3.3.1.2	A response covers many hours and places; the trip needs one. A missing forecast must not stop the estimate.	Share of fixtures resolved to the expected forecast hour (target 100%); runs where a missing response blocks the estimate (target 0).	Fixture responses with several hours and places, a late response and no response. (Test)

4 Summary

The first-level physical architecture of EcoCommute has eight subsystems, C.0.0 Operations, C.0.1 Entry, C.0.2 Reference Data, C.0.3 Estimation, C.0.4 Records, C.0.5 Recommendation, C.0.6 Presentation and C.0.7 Reporting, one for each first-level function. Their 24 conduits, 11 to external assets and 13 between subsystems, transfer all 22 first-level input/outputs, and the I2 diagram shows the coupling is low. The most complex function, F.2.2 Manage Reference Data, was decomposed into four functions performed by four modules, with two new internal flows that close the baseline scenario. IDEF0 diagrams with mechanisms now exist for three levels, with no tunneled or unconnected input, control or output arrows.

The analysis against the functional architecture found no violation; it flagged IO.2 reaching three subsystems directly, and the hub role of Entry and Records, for detailed design. Of three alternatives, the functional subsystems scored 4.35 against 2.75 and 3.25. For requirements, 11 design output requirements cover all leaf functions, each with parent, rationale, MOP and verification; the component specification for C.0.2 has 27 requirements, nine of them flowed down from the system-wide requirements by equivalence (4), apportionment (4) and synthesis (1) with budgets that sum to the system values.

Open items: the budgets, the November 13, 2026 integration date, the four-week pilot window (the measure of requirement 2.1.6 was moved from week eight to week four in model 11.3) and the 3-second and group-size-5 values remain educated guesses or proposals pending sponsor confirmation; the third level of F.2.2.1 is planned for the final assignment; place suggestion, which model 11.0 allocated to F.2.2, needs a fifth second-level function in the next iteration; and the repository code layout needs the allocation of the walking-skeleton table applied in the product build.

Appendix A. Model Assessment Check

Table 33 checks the work against the First-Level IDEF0 Assessment Criteria of Chapter 8 and Table 34 against the Second-Level Form Architecture and Component Specification Criteria of Chapter 9.

Table 33. First-level IDEF0 assessment criteria (Chapter 8).

Criterion	Status	Evidence
Each first-level function is performed by exactly one appropriately named asset	Met	Table 11; Figure 7
Top-level system is decomposed by the subsystems on the first-level IDEF0, one to one	Met	Figure 11
No tunnels on the Metafunction or first-level IDEF0	Met	Sections 2.1.2.1 and 2.1.3.3
Universe asset diagram shows conduits that transfer input/outputs consistent with the Metafunction	Met	Figure 13
System asset diagram shows conduits that transfer input/outputs consistent with the first-level IDEF0	Met	Figure 14; Table 18
System-wide requirements added to the system requirements document	Met	Section 3.4 of document 3 (17 leaf requirements, reviewed here)
Leaf-level system-wide requirements satisfied by the system asset	Met	All 17 leaf requirements of 3.4 satisfied by C.0 in model 11.3

Table 34. Second-level form architecture and component specification criteria (Chapter 9).

Criterion	Status	Evidence
Each second-level function is performed by exactly one asset	Met	Table 12; Figure 8
No tunnels on the Metafunction, first-level or second-level IDEF0	Met	Section 2.1.3.3
Component input, output and functional requirements read "The [named] subsystem shall ..."	Met	Table 25; Table 32
Every input, control and output of the second-level IDEF0 has an input or output requirement	Met	5.1.1 to 5.1.3 and 5.2.1 to 5.2.3 for IO.12, IO.5, IO.6, IO.4, IO.14, IO.15
Every second-level function has exactly one functional requirement	Met	5.4.1 to 5.4.4
An external interface requirement exists for each external system the component interacts with	Met	5.3.1 and 5.3.2 (Open-Meteo), 5.3.3 (Emission Factor Publisher)
A sample system-wide component requirement exists for every pertinent system-wide requirement	Met	Table 26 (9); omissions justified in Table 28
Flow-down has a rationale and a stated method	Met	Table 26
System-wide requirements are satisfied by the pertinent subsystem asset	Met	5.5.1 to 5.5.9 satisfied by C.0.2 in model 11.3

Appendix B. Model Baseline and Figure Sources

Model baseline. The architecture of this section is model 11.3 of the Innoslate project. Its lineage is: 10.8 (function architecture of Chapters 6 and 7); 10.9 (web flow: trip planner UC.3, Route Service X.6); 11.0 (accounts, bus timetable, vehicles, carpool and place suggestions: UC.4, X.7); 11.2 (this section: first-level subsystems C.0.0 to C.0.7, modules C.0.2.1 to C.0.2.4, second-level functions F.2.2.1 to F.2.2.4, input/outputs IO.23 and IO.24, 24 first-level and two second-level conduits, documents 4 and 5 with their requirements, and the trace of every leaf system-wide requirement to C.0); 11.3 (adds the two Route Service interface requirements 5.3.7 and 5.3.8). Model 11.2 was imported into Innoslate and Innoslate Intelligence reported a pass rate of 74% with no errors and warnings only; 11.3 adds two requirements to it. The walking-skeleton modules C.1 to C.4 remain in the model as performers of the UC.3 and UC.4 actions but are detached from C.0. The export is docs/model/EcoCommute_11.3_C8-C9_physical_architecture.xml and the scripts that built the report and the model are in docs/ch8-ch9/build/. Two requirement texts were changed in the model: 3.4.1.2 and 2.3.4 now read December 18, 2026, and 2.1.6 measures weekly active use after four weeks (it read eight); both changes need sponsor confirmation.

Table 35. Source of every figure.

Figure	Source
Figure 1	Innoslate screenshot, model 10.8 (Chapters 6 and 7)
Figure 2	Innoslate screenshot, model 11.2
Figure 3	Innoslate screenshot, model 10.8 (Chapters 6 and 7)
Figure 4	Drawn from the model data (Innoslate cannot show internal interactions)
Figure 5	Drawn from the model data (Innoslate cannot show internal interactions)
Figure 6	Innoslate screenshot, model 11.2
Figure 7	Innoslate screenshot, model 11.2
Figure 8	Innoslate screenshot, model 11.2
Figure 9	Innoslate screenshot, model 11.2
Figure 10	Innoslate screenshot, model 11.2
Figure 11	Innoslate screenshot, model 11.2
Figure 12	Innoslate screenshot, model 11.2
Figure 13	Innoslate screenshot, model 11.2
Figure 14	Innoslate screenshot, model 11.2
Figure 15	Innoslate screenshot, model 11.2
Figure 16	Innoslate screenshot, model 11.2
Figure 17	Chart of Table 21

Figure	Source
Figure 18	Innoslate screenshot, model 11.2